НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

Навчально-науковий інститут атомної та теплової енергетики
Кафедра цифрових технологій в енергетиці

«На правах рукопису»
УДК_____

«До захисту допущено»
Завідувач кафедри
_____ Наталія АУШЕВА

«___»_____2022р.

Магістерська дисертація

зі спеціальності - 122 Комп'ютерні науки
за освітньо-професійною програмою магістерської підготовки - Комп'ютерний моніторинг та геометричне моделювання процесів та систем

на тему Адаптивне управління автономним роботом на базі контролера STM32

Виконав: студент 2 курсу, групи ТР-11мп
Лесик Максим Олександрович_____ _____
(прізвище, ім'я, по батькові) (підпис)

Науковий керівник доц. к.т.н. Крячок О.С._____ _____
(посада, вчене звання, науковий ступінь, прізвище та ініціали) (підпис)

Консультант _____ _____ _____
(назва розділу) (вчені ступінь та звання, прізвище, ініціали) (підпис)

Рецензент_____ _____
(посада, вчене звання, науковий ступінь, прізвище та ініціали) (підпис)

Засвідчую, що у цій магістерській дисертації немає запозичень з праць інших авторів без відповідних посилань.
Студент _____
(підпис)

Київ — 2022

Національний технічний університет України
«Київський політехнічний інститут ім. Ігоря Сікорського»

Навчально-науковий інститут атомної та теплової енергетики
Кафедра цифрових технологій в енергетиці
Рівень вищої освіти другий, магістерський
За освітньою програмою «Комп'ютерний моніторинг та геометричне моделюванняпроцесів та систем»

Спеціальності - 122 Комп'ютерні науки

ЗАТВЕРДЖУЮ
Завідувач кафедри
_____ Наталія АУШЕВА
(підпис)
«____»_____2022р.

З А В Д А Н Н Я
НА МАГІСТЕРСЬКУ ДИСЕРТАЦІЮ СТУДЕНТУ

Лесику Максиму Олександровичу
(прізвище, ім'я, по батькові)

1. Тема дисертації Адаптивне управління автономним роботом на базі контролера STM32
Науковий керівник Крячок Олександр Степанович, к.т.н., доц.
(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)
затверджені наказом по університету від «07» листопада 2022 року № 4067-с
2. Строк подання студентом дисертації 7 грудня 2022 року
3. Вихідні дані до роботи плата на базі контролера STM32, конструктор для створення робота
4. Перелік питань, які потрібно розробити
   - проаналізувати існуючі методи адаптивного управління роботом;
   - обґрунтувати, які методи найкраще підходять для вирішення задачі магістерської роботи;
   - ознайомитись з функціональними можливостями сучасних засобів, що використовуються у робототехніці;
   - спроєктувати автономного дрона;
   - розробити програмне забезпечення управління роботом.
5. Орієнтований перелік ілюстративного матеріалу презентація на тему «Адаптивне управління автономним роботом на базі контролера STM32» складається з актуальності, мети та завдання дослідження, об'єкту та предмету дослідження, наукової новизни роботи, засобів розробки, структурної схеми, діаграми прецедентів та роботи з програмним продуктом.
6. Орієнтований перелік публікацій
1. Лесик М.О., Крячок О.С. Адаптивне управління автономним роботом на базі контролера STM32 / IX Міжнародна науково-практична конференція «MODERN RESEARCH IN WORLD SCIENCE», 28-30.11.2022 Львів, Україна, с. 525-529. URL: https://sci-conf.com.ua/wp-content/uploads/2022/12/MODERN-RESEARCH-IN-

WORLD-SCIENCE-28-30.11.2022.pdf

7. Дата видачі завдання «___» жовтня 2021р.

КАЛЕНДАРНИЙ ПЛАН

№ з/п	Назва етапів виконання дипломної роботи	Термін виконання етапів роботи	Примітки
1.	Вивчення та аналіз задачі	01.09.22р.-14.09.22р.	
2	Розробка архітектури та загальної структури системи	15.09.22р.-21.09.22р.	
3.	Розробка бази даних системи	22.09.22р.-28.09.22р.	
4.	Програмна реалізація системи	29.09.22р.-19.10.22р.	
5.	Оформлення пояснювальної записки	20.10.22р.-20.11.22р.	
6.	Захист програмного продукту	20.10.22р.	
7.	Передзахист	21.11.22р.-30.11.22р.	
8.	Захист	19.12.22р.-23.12.22р.	

Студент _____ Лесик М.О.
(підпис) (прізвище та ініціали)

Науковий керівник _____ Крячок О.С.
(підпис) (прізвище та ініціали)

РЕФЕРАТ

Магістерська дисертація складається зі вступу, шести розділів, висновку, та 1 додатку, переліку посилань з 30 найменувань, і містить 30 рисунків, 22 таблиць. Повний обсяг магістерської дисертації складає 84 сторінки, з яких перелік посилань займає 3 сторінки, та додатки 7 сторінок.

Актуальність теми. З розвитком технологій роботи стали життєво важливим інструментом у повсякденному житті людей. Важливим аспектом створення роботів є те, що вони корисні та ефективні. Вони допомагають зробити життя людей легшим, простішим і комфортнішим. Вони також замінюють людську робочу силу для виконання нудної, монотонної, брудної чи небезпечної роботи. Вони привели людство вперед, будучи частиною відкриття нової інформації. Допомога у завданнях, надання допомоги учням і вчителям у навчанні та догляд за літніми людьми є одними з ролей, які роботи виконують у повсякденному житті.

Метою розробки роботизованої системи є створення програмного продукту та машини, що дасть змогу, виконувати певні розвідувальні задачі та проходження певного маршруту самостійно або за допомогою користувача.

Були сформовані наступні завдання дослідження:

- проаналізувати існуючі методи адаптивного управління роботом;
- обґрунтувати, які методи найкраще підходять для вирішення задачі магістерської роботи;
- ознайомитись з функціональними можливостями сучасних засобів, що використовуються в робототехніці;
- спроєктувати дрон;
- розробити програмне забезпечення для управління роботом.

Об’єктом дослідження є інформаційні технології системи управління автономним роботом.

Предметом дослідження є інформаційні засоби і методи робототехніки.

Науковою новизною проєкту є створення покращеної та більш зручної моделі управління автономним роботом.

Практичне значення одержаних результатів. Розробка автономного робота для проходження певного маршруту самостійно або під керуванням користувача, та створення програмного продукту для управління даним дроном.

Апробація результатів дисертації

Основні положення роботи доповідались і обговорювались на:

IX Міжнародна науково-практична конференція «MODERN RESEARCH IN WORLD SCIENCE», 28-30.11.2022 Львів, Україна

Ключові слова: *СИСТЕМА УПРАВЛІННЯ, АВТОНОМНИЙ РОБОТ ДРОН, РОБОТОТЕХНІКА.*

ABSTRACT

The master's dissertation consists of an introduction, five chapters, a conclusion, and 1 application, a listof references from 30 titles, and contains 30 figures, 22 tables. The full volumeof the master's dissertation is 84 pages, of which the list of references is 3 pages and appendices of 7 pages.

Topicality. With the development of technology, robots have become a vital tool in people's daily lives. An important aspect of creating robots is that they are useful and effective. They help make people's lives easier, simpler and more comfortable. They also replace human labor to perform boring, monotonous, dirty or dangerous work. They have led humanity forward by being part of the discovery of new information. Helping with military tasks, helping students and teachers with their studies, and caring for the elderly are some of the roles that robots play in everyday life.

The purpose of developing a robotic system is to create a software product and a machine that will enable you to perform certain reconnaissance tasks and travel a certain route independently or with the help of a user.

The following research tasks were formulated:

– analyze existing methods of adaptive robot management;

- justify which methods are best suited for solving the problem of the master's thesis;

- get acquainted with the functionality of modern tools used in robotics;

– design a drone;

– to develop software for controlling the robot.The object of the research is information technologies for environmental monitoring.

The object of the study is information technologies of the autonomous robot control system.

The subject of research is informational means and methods of robotics.

The scientific novelty of the project is the creation of an improved and more convenient control model for an autonomous robot.

Practical significance of the obtained results. Development of an autonomous robot to travel a certain route independently or under the control of the user, and creation of a software product for controlling this drone.

Approbation of the results of the dissertation

The main provisions of the work were reported and discussed at:

IX International Scientific and Practical Conference «MODERN RESEARCH IN WORLD SCIENCE», November 28-30, 2022 Lviv, Ukraine

Keywords: *MANAGEMENT SYSTEM, AUTONOMOUS ROBOT DRONE, ROBOTIC TECHNOLOGY.*

ЗМІСТ

ВСТУП 10
1 СУЧАСНІ СИСТЕМИ УПРАВЛІННЯ АВТОНОМНИМИ БЕЗПІЛОТНИМИ АПАРАТАМИ 11
1.1 Актуальність безпілотних наземних транспортних засобів 11
1.2 Контроль систем управління 13
1.3 Основи систем управління 15
1.4 Типи систем управління 17
1.5 Керування роботом-маніпулятором 20
2 ОГЛЯД КОНТРОЛЕРІВ ДЛЯ СИТЕМ УПРАВЛІННЯ РОБОТИЗОВАНИМИ ПЛАТФОРМАМИ 21
2.1 Мікроконтролери 21
2.2 Пам’ть мікроконтролерів 23
2.3 Процесорне ядро мікроконтролерів 29
2.4 Переривання мікроконтролерів 33
2.5 Контролер STM32 35
3 ЗАСОБИ РОЗРОБКИ 39
3.1 Мова програмування C++ 39
3.2 Платформа на базі Arduino 41
3.3 Платформа на базі STM32 42
3.4 Середовище розробки Arduino IDE 43
4 ОПИС ПРОГРАМНОЇ РЕАЛІЗАЦІЇ 45
4.1 Діаграма варіантів використання 45
4.2 Електронна схема 46
4.3 Структурна схема 47
4.4 Опис програмного коду 48
5 РОБОТА КОРИСТУВАЧА З ПРОГРАМНОЮ СИСТЕМОЮ 53
5.1 Опис системних вимог 53
5.2 Робота користувача з продуктом 53

6 РОЗРОБЛЕННЯ СТАРТАП ПРОЄКТУ 58
6.1 Опис ідеї проєкту 59
6.2 Технологічний аудит ідеї проєкту 60
6.3 Аналіз ринкових можливостей запуску стартап-проєкту 60
6.4 Розроблення ринкової стратегії 67
6.5 Розроблення маркетингової програми стартап-проєкту 69
6.6 Висновки до розділу 73
ВИСНОВКИ 74
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ 75
ДОДАТОК А 78

ВСТУП

З розвитком технологій роботи стали життєво важливим інструментом у повсякденному житті людей. Важливим аспектом створення роботів є те, що вони корисні та ефективні. Вони допомагають зробити життя людей легшим, простішим і комфортнішим. Вони також замінюють людську робочу силу для виконання нудної, монотонної, брудної чи небезпечної роботи. Вони привели людство вперед, будучи частиною відкриття нової інформації. Допомога у завданнях, надання допомоги учням і вчителям у навчанні та догляд за літніми людьми є одними з ролей, які роботи виконують у повсякденному житті.

Роботи змінили збройні сили безліччю способів, привносячи нові ідеї на поле бою. Впровадження роботів в армію призвело до реконструкції систем захисту, що є важливим завданням для країни. Роботи також змінили погляди людей на сутички, змусивши їх більше схилятися до неї, оскільки використання штучного інтелекту означало б менше жертв. Під час бою війська зазвичай надають машинам, які використовуються для розвідки та видобутку корисних копалин, людські характеристики. В Афганістані та Іраку американські солдати ставляться до них як до партнерів, нагороджуючи їх медальйонами за завершені операції (Bond, 2009). Іншими словами, роботи використовуються в багатьох завданнях у військах, наприклад, замінюють окремих людей у завданнях і допомагають зробити роботу ефективнішою для всіх. Таким чином, роботи подали нове світло на сутички, змінивши способи боротьби людей у минулому.

Роботи також допомагають у сфері освіти. Вони досить популярні в деяких регіонах, наприклад в Японії; а деякі школи та університети використовують роботів, які виконують роль учителів або допомагають у процесі навчання.

1 СУЧАСНІ СИСТЕМИ УПРАВЛІННЯ АВТОНОМНИМИ БЕЗПІЛОТНИМИ АПАРАТАМИ

У розділі обґрунтовано нормативно-методичну основу формування системи управління автономним роботом. Визначено необхідність використання створення такої системи. Проведено аналіз схожих систем .

1.1 Актуальність безпілотних наземних транспортних засобів

Безпілотний наземний транспортний засіб – це комплексна інтелектуальна система, яка об’єднує сприйняття навколишнього середовища, місцезнаходження, навігацію, планування шляху, прийняття рішень і керування рухом. Він поєднує високі технології, включаючи інформатику, злиття даних, машинне бачення, глибоке навчання тощо, щоб задовольнити реальні потреби для досягнення заздалегідь визначених цілей [1].

У сфері цивільного застосування безпілотні наземні транспортні засоби в основному втілені в автономному водінні. Високоінтелектуальні моделі водіїв можуть повністю або частково замінити активний контроль водія. Крім того, безпілотні наземні транспортні засоби з датчиками можуть легко діяти як «транспортні засоби-зонди» та здійснювати зондування дорожнього руху для кращого обміну інформацією з іншими агентами в інтелектуальних транспортних системах. Таким чином, він має великий потенціал у зменшенні дорожньо-транспортних пригод і зменшенні заторів. У сфері бойового застосування він компетентний у таких завданнях, як отримання розвідданих, моніторинг і розвідка, транспортування та матеріально-технічне забезпечення, розмінування та розміщення саморобних вибухових пристроїв, забезпечення вогневої підтримки, передача зв’язку та медична передача на полі бою, що може ефективно допомагати у бойових діях.

Технологія сприйняття середовища включає одночасну локалізацію та відображення, семантичну сегментацію, виявлення транспортних засобів, виявлення пішоходів, виявлення дороги та багато інших аспектів. Серед різноманітних технологій, оскільки транспортні засоби є найчисленнішими та найрізноманітнішими цілями в середовищі водіння, як правильно ідентифікувати транспортні засоби, стало гарячою точкою дослідження для безпілотних наземних транспортних засобів. У цивільній сфері правильне виявлення дорожніх транспортних засобів може зменшити кількість дорожньо-транспортних пригод, створити більш повну картину безпеки і досягти кращої інтеграції з моделлю водія, тоді як у бойовій сфері правильне виявлення цілей транспортних засобів має велике значення для поля бою. розвідка, оцінка загроз і точна атака в сучасних сутичках.

Зазначимо, що штучний інтелект (ШІ) та робототехніка завжди були тісно пов'язані один з одним. Одним з важливих направлень ШІ досі вважається цілеспрямоване поведінка роботів (створення інтелектуальних роботів, здатних автономно здійснювати операції з досягненню цілей, поставлених людиною). Інтелектуальний робот як машина-виконавець повинен приймати завдання в загальній формі, а сам робот повинен мати можливість приймати рішення або планувати свої дії в невизначеній або складній обстановці, що розпізнається ним.

Для транспортного робота це означає, що система має самостійно обирати маршрут до наміченої мети, обходити перешкоди, які можуть виникати на шляху, вибирати оптимальні параметри переміщення – швидкість, положення корпусу просторі, прискорення на віражах, а також запобігати зіткненням і аварійним ситуаціям [2].

За минуле десятиліття з’явилося багато розробок, орієнтованих насамперед створення роботів для домашнього господарства, де потрібні досить складні інтелектуальні функції (рисунок1.1). Насамперед – це орієнтація та переміщення в обмеженому просторі з мінливою обстановкою

(Предмети в будинку можуть змінювати своє місце розташування), відкривання та закривання дверей під час переміщення по будинку [3].



Рисунок 1.1 – Робот пилосос для домашнього користування

Роботи зі створення домашніх роботів ведуть багато великих університети світу, часто на замовлення військових. Це пов’язано з можливістю використання розроблених технологій у різних областях, і в першу чергу – для створення роботів.

1.2 Контроль систем управління

Контроль регулює поведінку інших підсистем. Він повинен мати всі знання про поточні та наступні етапи всіх систем, якими він керує. Система управління має два контури – відкритий і закритий. Роботехнічні системи зараз широко впроваджуються в «розумне виробництво».

Робототехніка знайшла одне з перших застосувань у верстатобудівній промисловості. Тоді роботи були сконструйовані таким чином, щоб мати

механізм із жорсткою рукою з кожним суглобом, який керувався окремо як лінійна система SISO. Крім того, існувала система «точка-точка» для таких завдань, як передача матеріалу, і система безперервного відстеження шляху для виконання таких завдань, як зварювання та фарбування. У сучасному світі системи керування роботами значно вдосконалилися з точки зору техніки, а системи зору розширили призначення роботів. Багаторівнева система керування допомагає збільшити можливості та гнучкість робота [4].

Командна робота людини та робота відіграє вирішальну роль у підвищенні ефективності роботи механізму. Багато складних завдань виконуються в режимі реального часу автономними мобільними роботами. Сучасні роботизовані системи керування зробили революцію у виробничій галузі, зробивши її надзвичайно універсальною та зручною для користувача. PID контролери використовуються більш ніж у дев'яносто відсотках промислових контролерів. Це головним чином тому, що він допускає різні правила налаштування, які були запропоновані у відповідній літературі, і більшість цих контролерів можна налаштувати на місці за допомогою цих правил. Деякі контролери також можуть мати можливості онлайнового автоматичного налаштування та методи автоматичного налаштування. У промисловості використовуються імпровізовані форми PID регулювання, такі як PID регулювання з кількома ступенями свободи та I-PD-регулювання.

Відбувся величезний прогрес у галузі штучної нейронної мережі, теорії нечіткої логіки, штучного інтелекту та еволюційних обчислювальних методів, таких як генетичний алгоритм, оптимізація частинок тощо. усі інтелектуальні обчислювальні методи, які надали нам новий набір рішень для проблем у різних системах керування.

Лінійний квадратичний регулятор (LQR) – це сучасний метод керування зі зворотним зв'язком, який використовує підхід простору станів для аналізу систем. На даний момент LQR використовується з комбінацією PID і FLC, щоб допомогти керувати двоколісним роботом, що самобалансується (рисунок 1.2).

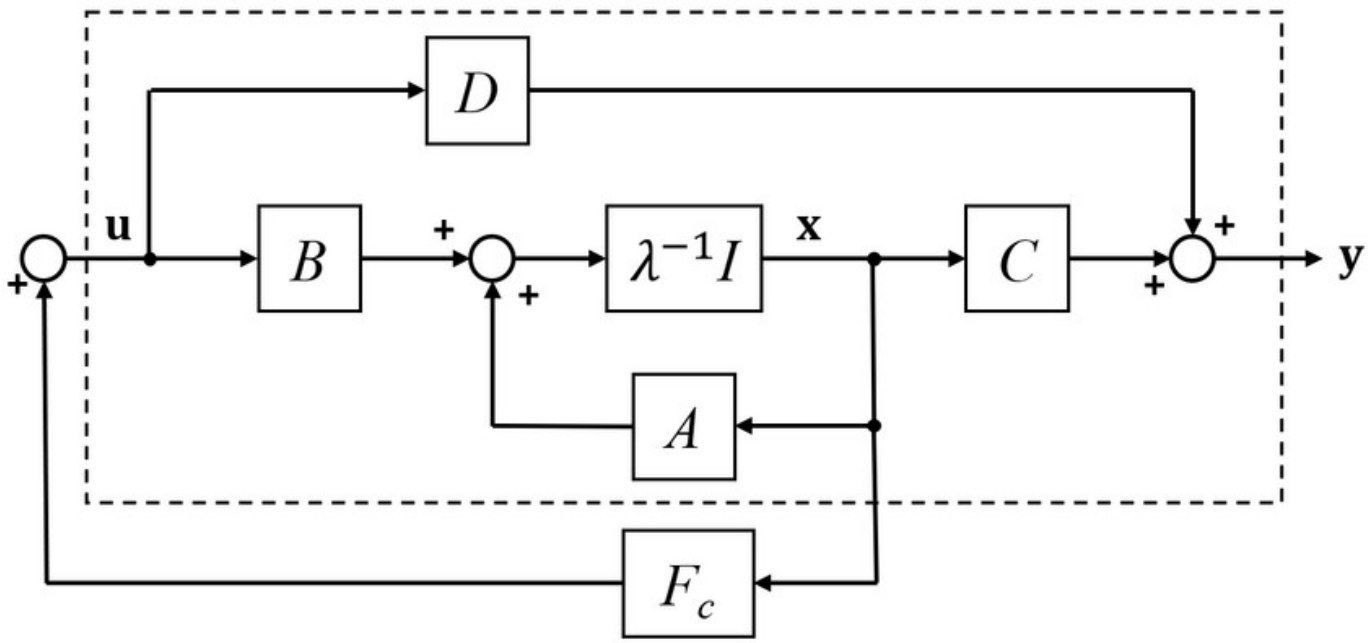


Рисунок 1.2 – Приклад лінійно-квадратичного регулятора

1.3 Основи систем управління

Першою є відкритий цикл. Структури, в яких контрольна діяльність не залежить від врожайності, відомі як керування з відкритим колом каркасу. Так би мовити, продуктивність системи керування незамкнутим колом не визначена чи підкріплена для кореляція з Input. Фундаментальним недоліком цієї системи управління є те, що керована змінна чутлива до змін тривожних впливів. Однією з приземлених ілюстрацій є пральна машина. Операція обливання, миття та промивання в пральній машині відбувається в приміщенні періоду [5].

Друга основа це закритий цикл. Системи керування зі зворотним зв'язком найчастіше називають системами керування із замкнутим контуром. Терміни керування замкнутим контуром і керування зі зворотним зв’язком є синонімами. У замкнутій системі керування сигнал помилки приведення в дію, який є розрахованою різницею між вхідним сигналом і сигналом зворотного зв’язку, подається на контролер, щоб зменшити помилку та стабілізувати

вихід системи шляхом приведення його до бажаного значення. Контроль із замкнутим циклом зменшує помилки системи, завжди використовуючи дію керування зворотним зв’язком (рисунок 1.3).

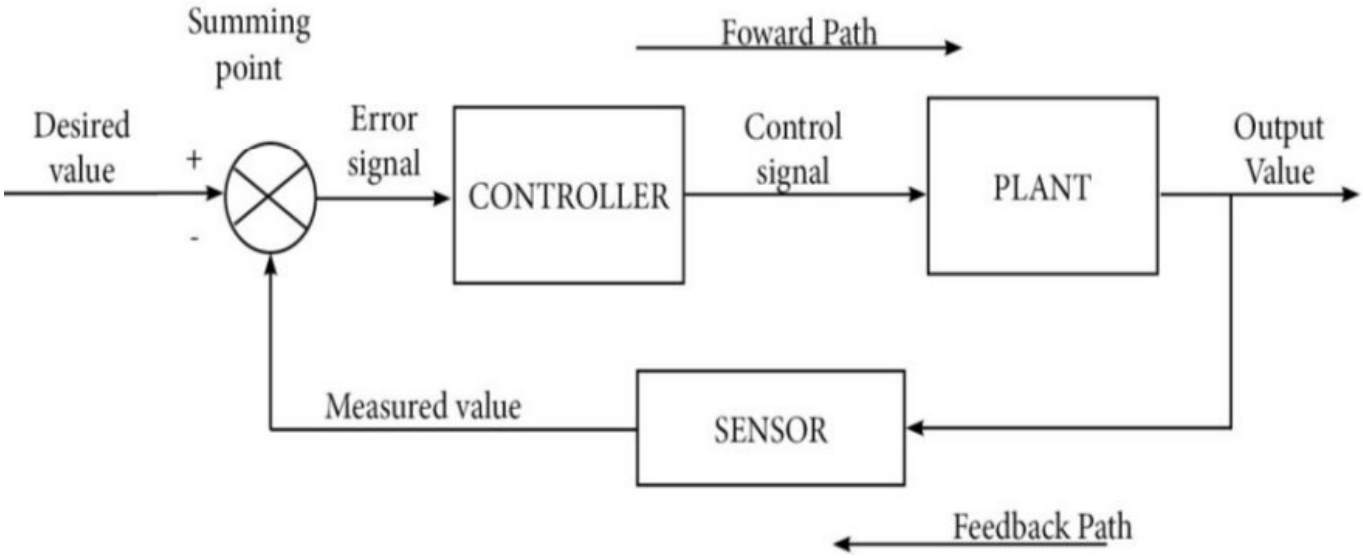


Рисунок 1.3 – Система з закритим циклом

Вище наведена блок-схема, що представляє замкнуту систему, пояснює використання принаймні одного датчика та контролера на основі виходу установки. Установка і контролер знаходяться в прямому напрямку, і датчик передає це на лінію зворотного зв’язку. Вихід, отриманий установкою, порівнюється з бажаним значенням у точці підсумовування. Різниця надсилається до контролера, який видає керуючий сигнал для керування установкою, поки бажане значення та вихідне значення не зрівняються.

Третьою є лінійні системи. Лінійні системи – це системи, які дотримуються основних законів суперпозиції. Цей закон стверджує, що коли дві різні функції застосовуються одночасно, дана відповідь є наслідком двох індивідуальних реакцій системи керування. Це полегшує розрахунок відповіді на кілька входів лінійних систем шляхом розгляду одного входу в певний момент часу та додавання їхніх виходів. Саме цей закон створює складні розв’язки лінійного диференціального рівняння з можливих простих розв’язків [6].

1.4 Типи систем управління

Для керування роботами тут представлені такі методи керування.

Відносний основний підлеглий (PID) контроль обробляє відмінні справжнє керування проблемами в усіх складних, але найкращим чином. Комфорт із трьома мітками (тобто P, I та D) вирішує проблеми за допомогою безперервних станів і тимчасових реакцій. Розвиток автоматизованого прогресу зробив систему керування модифікованою можливою, що пропонує розташування варіантів для контрольної ділянки, ті самі контролери можуть полегшити простоту, пристосування, доречність та чітку цінність ПІД-регулятора (рисунок 1.4).

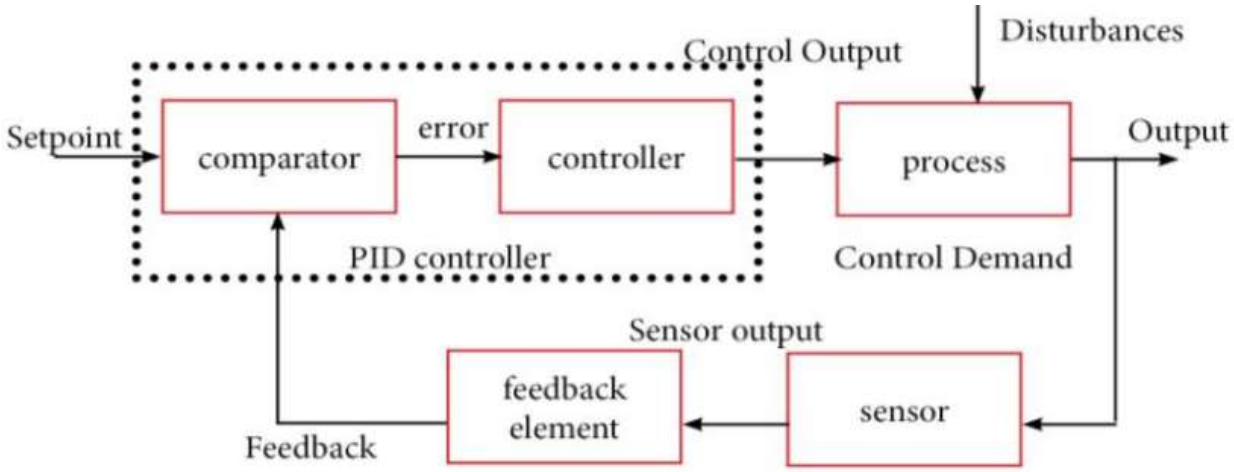


Рисунок 1.4 – Система з закритим циклом двоколісного робота

Двоколісний робот – це хитка, непряма та багатогранна рама з незамкнутим колом, що робить універсальний ПІД-регулятор найбільш придатним для застосування. Фундаментальна функція цього універсального робота полягала в тому, щоб налаштувати робота, поки він просувається до місця призначення. Робиться постріл по керівній лінії зміненого маятника. Універсальний робот за своєю суттю ненадійний, може впасти зі свого верхнього вертикального положення без сторонньої допомоги. Він має численні параметри, займає менше місця та може легко досліджувати

ландшафти. Ці контролери не є енергійними, тому вони мають внутрішні та зовнішні тривожні впливи. Наслідки досліджень показують, що техніка підвищила міцність. Це нововведення є перспективним у широких колах застосування.

Лінійно-квадратичний контролер – це процедура в сучасному управлінні, у якій використовується простір станів для розгляду такої системи. Методологія схеми MIMO є ідеальною стратегією управління лінійно-квадратичним контролером. Ідея полягає в тому, щоб замінити цикл архітектора на поштові області, які використовуються як частина повного введення стану, на акценти на компонентах у функції витрат [7].

Прямий квадратичний контролер – це одна з ідеальних стратегій керування, яка враховує умови динамічної структури та керуючої інформації для визначення ідеального вибору керування. Ідеальний контролер критики стану з огляду на стратегію лінійно-квадратичного контролера був призначений для двомоторної багато інформаційної багатофункціональної структури.

TRMS – це нелінійна структура з двома ступенями можливості, крім того, перехресними зв’язками. Призначення двомоторної системи з декількома входами і виходами полягає в тому, щоб керувати польотом гелікоптера [8].

Радіочастотна ідентифікація забезпечує бездротовий доступ до системи керування (рисунок 1.5). Підсистема складається з трьох компонентів: RFID-мітки, пристрою для зчитування міток і обчислювального пристрою для керування інформаційною базою даних. Зчитувач вловлює сигнали RFID-міток, і ця інформація обробляється та перевіряється обчислювальним пристроєм. За допомогою радіочастотної ідентифікації було досягнуто широкого спектру застосувань автоматизованого відстеження та бездротового зв’язку. Радіочастотна ідентифікація не є недавнім відкриттям, військові програми та програми безпеки реалізували комплексну інтеграцію активної

системи RFID. Пасивні системи радіочастотної ідентифікації, які є досить дешевими порівняно з активними, показали багато перспектив на ринку.

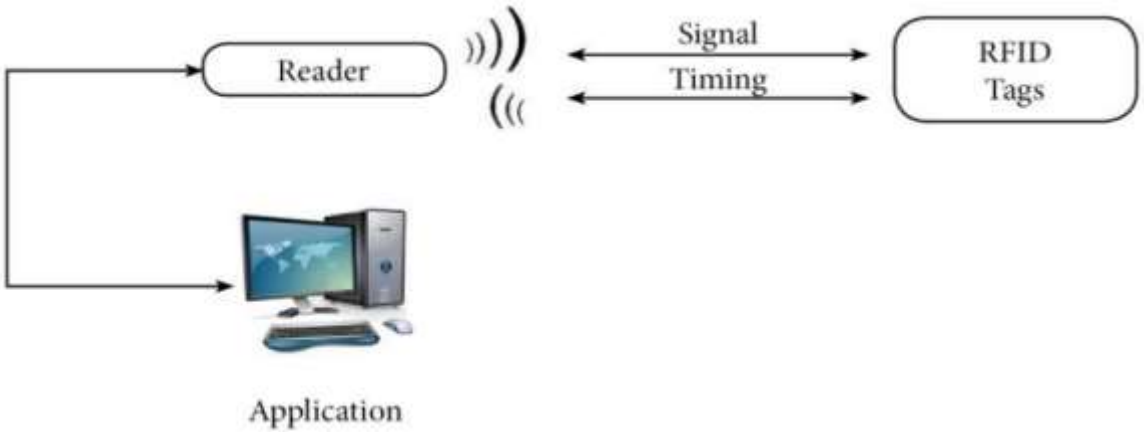


Рисунок 1.5 – Приклад радіочастотної системи

Деталі надходять у підсистеми обробки через ворота радіочастотної ідентифікації. Станція радіочастотної ідентифікації не тільки сканує на наявність деталей, але й постійно оновлює інформацію про теги, а також параметри завантаження та розвантаження станції. Шлях, яким пройде деталь, залежить від її RFID-мітки та ідентифікаторів станції, що призводить до контрольованого потоку деталей через конвеєрну систему. Процеси підстанції управляються за допомогою високопродуктивних промислових ПК, і коли деталь буде готова, вона повертається на конвеєри. Зміни в замовленнях спочатку обробляються, а потім вторинні інструкції передаються до високопродуктивних промислових комп'ютерів відповідної станції з HML. Високопродуктивний промисловий комп'ютер порівнює дані тегів зі змінами замовлення, отриманими через HMI, і приймається рішення для визначення потоку частин потім-після, тобто деталь, яку потрібно обробити на поточній або іншій станції. Коригування вносяться в дані радіочастотної ідентифікації, які допомагають шлюзу радіочастотної ідентифікації створювати загальні маршрути для конвеєрних систем, щоб забезпечити ефективну логістику деталей. Гнучка природа радіочастотної ідентифікації робить її здатною до застосування контактів на великій відстані, зберігаючи легкість

програмування, керування зберіганням і навіть сканування поза межами прямої видимості.

1.5 Керування роботом-маніпулятором

Динаміка приводу, різні системні невизначеності та гнучкість з’єднань беруться до уваги, коли йдеться про проблему керування роботами-маніпуляторами. Тепер кажуть, що ці невизначеності змінюються в часі, але їх межі, що змінюються в часі, недоступні, що робить адаптивне керування практичним через його різний характер [9].

Динаміка робота дуже нелінійна, а конструкція складна. На основі варіаційних меж визначаються контрольні параметри, що підтверджує високу стабільність замкнутої системи.

У випадку гнучкого шарнірного робота конструкція контролера підвищує продуктивність управління, і вона дуже складна порівняно з жорстким роботом. Гнучкість з’єднань виявилася дуже важливим компонентом динаміки й контролю робота, але раніше вона була відома як обмежуючий фактор для продуктивності маніпулятора.

Найновішою тенденцією в маніпуляторах є дистанційне керування роботами. У цьому роботами-маніпуляторами керує віддалений оператор-людина. Були виявлені проблеми затримки між сенсорним зворотним зв’язком і командою, наданою маніпулятору оператором. Щоб подолати залежність від затримки, було використано керування на основі пасивності та запроваджено телеоператорську систему «ведучий-підлеглий», яка діяла як з’єднання мереж з двома портами [10].

2 ОГЛЯД КОНТРОЛЕРІВ ДЛЯ СИСТЕМ УПРАВЛІННЯ РОБОТИЗОВАНИМИ ПЛАТФОРМАМИ

У розділі обґрунтовано нормативно-методичну основу формування системи управління автономним роботом. Визначено необхідність використання створення такої системи. Розглянуті методи та підходи проєктування робототехніки.

2.1 Мікроконтролери

Мікроконтролер — це однокристальний комп'ютер, спеціально виготовлений для програм керування вбудованим комп'ютером. Ці пристрої дуже недорогі, і їх можна дуже легко використовувати в програмах цифрового керування. Більшість мікроконтролерів мають вбудовані схеми, необхідні для програм керування комп'ютером. Наприклад, мікроконтролер може мати аналого-цифрові перетворювачі, щоб можна було дискретизувати зовнішні сигнали. Вони також мають паралельні порти вводу-виводу, щоб цифрові дані можна було зчитувати або виводити з мікроконтролера. Деякі пристрої мають вбудовані цифро-аналогові перетворювачі, і вихідний сигнал перетворювача можна використовувати для керування установкою через виконавчий механізм (наприклад, підсилювач). Мікроконтролери також можуть мати вбудований таймер і логіку переривань. Використовуючи таймер або засоби переривання, ми можемо запрограмувати мікроконтролер на точну реалізацію алгоритму керування [11].

Мікроконтролери традиційно програмуються з використанням мови асемблера цільового пристрою. Як наслідок, мови складання мікроконтролерів, вироблених різними фірмами, абсолютно різні, і користувачеві доводиться вивчати нову мову, перш ніж мати можливість

програмувати новий тип пристрою. У наш час мікроконтролери можна програмувати за допомогою мов високого рівня, таких як BASIC, PASCAL або C. Мови високого рівня пропонують кілька переваг порівняно з мовою асемблера:

- Простіше розробляти програми, використовуючи мову високого рівня.
- Обслуговування програми набагато легше, якщо програма розроблена з використанням мови високого рівня. Тестувати програму, розроблену мовою високого рівня, набагато легше.
- Мови високого рівня більш зручні для користувача та менш схильні до помилок. Легше документувати програму, розроблену з використанням мови високого рівня.

Крім перерахованих вище переваг, мови високого рівня мають деякі недоліки. Наприклад, довжина коду в пам'яті зазвичай більша, коли використовується мова високого рівня, і програми, розроблені з використанням мови асемблера, зазвичай працюють швидше, ніж ті, розроблені з використанням мови високого рівня.

У наш час комп’ютерне обладнання є дуже швидким, і керуючі комп’ютери, як правило, програмуються за допомогою мови високого рівня. Використання мови асемблера зарезервовано для дуже спеціальних і критичних до часу програм, таких як швидкі драйвери пристроїв у реальному часі. С — популярна мова, яка використовується в більшості додатків керування комп’ютером. Це потужна мова, яка дозволяє програмісту виконувати операції низького рівня без необхідності використання мови асемблера [12].

Вимоги до програмного забезпечення для керуючого комп’ютера можна підсумувати таким чином:

- можливість зчитування даних з вхідних портів; можливість передачі даних на вихідні порти;
- внутрішня передача даних і математичні операції;
- засоби переривання таймера для синхронізації алгоритму контролера.

Усім цим вимогам може задовольнити більшість цифрових комп’ютерів, і, як наслідок, більшість комп’ютерів можна використовувати як контролери в цифрових системах керування. Важливим моментом є те, що використання міні-комп’ютера для керування швидкістю двигуна, наприклад, невиправдано та нерентабельно. Мікроконтролер набагато більше підходить для такого типу програм керування. З іншого боку, якщо є багато вхідних даних і багато виходів, і якщо потрібно також забезпечити контрольні завдання, тоді використання міні-комп’ютера може бути легко виправданим.

Алгоритм контролера в комп’ютері реалізований як програма, яка безперервно виконується в циклі, який виконується на початку кожного часу вибірки. Усередині циклу зчитується бажане еталонне значення, також зчитується фактичний вихід заводу та обчислюється різниця між бажаним значенням і фактичним значенням. Це формує сигнал помилки. Потім реалізується алгоритм керування та обчислюється вихід контролера для цього моменту вибірки. Цей вихід надсилається на цифро-аналоговий перетворювач, який генерує аналоговий еквівалент бажаної керуючої дії. Цей сигнал потім подається на виконавчий механізм, який, у свою чергу, приводить установку в потрібну точку.

2.2 Пам’ять мікроконтролерів

Таким чином, можна зробити очевидне розрізнення типів пам'яті відповідно до їх функції:

Файл реєстрації: (зазвичай) відносно невелика пам'ять, вбудована в ЦП. Він використовується як блокнот для тимчасового зберігання значень, з якими працює ЦП - це можна назвати короткочасною пам’яттю ЦП.

Пам'ять даних: для довготривалого зберігання загальні процесори зазвичай використовують зовнішню пам'ять, яка набагато більша за файл реєстру. Дані, які там зберігаються, можуть бути короткочасними, але також можуть бути дійсними, доки працює ЦП. Звичайно, підключення зовнішньої

пам’яті до ЦП вимагає певних апаратних зусиль і, отже, спричиняє певні витрати. З цієї причини мікроконтролери зазвичай мають вбудовану пам’ять даних [13].

Пам'ять інструкцій: як і пам'ять даних, пам'ять інструкцій зазвичай є відносно великою зовнішньою пам'яттю (принаймні для звичайних процесорів). Насправді, з архітектурою фон Неймана це може бути навіть та сама фізична пам’ять, що й пам’ять даних. З мікроконтролерами пам'ять інструкцій також зазвичай інтегрована безпосередньо в MCU.

Це найбільш помітні способи використання пам’яті в ЦП або навколо нього. Однак пам'яті в ЦП більше, ніж це очевидно. Залежно від типу ЦП можуть бути конвеєрні регістри, кеші, різні буфери і так далі [14].

Щодо пам’яті, вбудованої в мікроконтролер: природно, розмір такої вбудованої пам’яті обмежений. Навіть гірше, часто неможливо розширити пам'ять ззовні (щоб зберегти простий дизайн). Однак, оскільки мікроконтролери найчастіше використовуються для відносно простих завдань і, отже, не потребують надмірних обсягів пам’яті, доцільно включити невеликий обсяг пам’яті даних і інструкцій на мікросхемі. Таким чином, загальна вартість системи значно зменшується, і навіть якщо пам’ять не розширюється, ви не обов’язково застрягли на цьому: різні члени сімейства MCU зазвичай надають різні обсяги пам’яті, тому ви можете вибрати певний MCU, який пропонує відповідний простір пам’яті.

Функціональне розрізнення типів пам’яті, зроблене вище, базується на способі використання пам’яті. З точки зору програміста, це має сенс. Однак розробники апаратного забезпечення чи чіпів зазвичай дивляться на пам’ять досить по-різному: вони вважають за краще розрізняти за фізичними властивостями електронних частин, з яких складається пам’ять. Найосновнішою відмінністю буде енергозалежна та енергонезалежна пам’ять. У цьому контексті непостійна означає, що вміст пам’яті втрачається, як тільки живлення системи вимикається.

Звичайно, існують різні способи реалізації будь-якого типу пам’яті. Тому розрізнення, засноване на фізичних властивостях, може бути більш детальним. Енергозалежна пам'ять може бути статичною або динамічною, і існує досить багато типів енергонезалежної пам'яті: ROM, PROM, EPROM, EEPROM, FLASH, NV-RAM. Давайте розглянемо їх уважніше [15].

Проблема тут полягає в тому, що енергонезалежна пам’ять зазвичай набагато повільніша, з нею складніше працювати та набагато дорожча. Хоча енергозалежна пам’ять вашого комп’ютера має час доступу в діапазоні наносекунд, деякі типи енергонезалежної пам’яті будуть недоступні протягом мілісекунд після запису в них одного поганого байта (рисунок2.1).

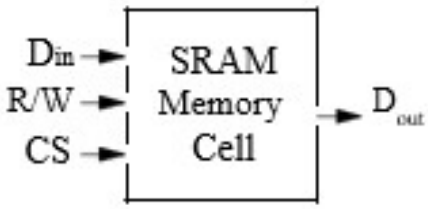


Рисунок 2.1 – Схема роботи пам’яті

Не враховуючи еру комп’ютерів до використання інтегральних схем, статична пам’ять з довільним доступом (SRAM) була першим типом енергозалежної пам’яті, який отримав широке застосування. Мікросхема SRAM складається з масиву комірок, кожна з яких здатна зберігати один біт інформації. Для зберігання частини інформації використовується так званий тригер, який в основному складається з шести транзисторів.

Дані в Din При цьому введенні комірка приймає один біт даних для збереження. Data Out Dout Як випливає з назви, цей вихід відображає біт, який зберігається в комірці. Читання/запис R/W Через логічне значення на цьому вході вказується тип доступу: 0 означає, що в комірку має бути записано, тобто поточний стан Din має зберігатися в комірці. 1 означає, що клітинка має бути прочитана, тому для Dout має бути встановлено збережене значення. Cell Select CS Поки цей вхід дорівнює логічному 0, комірка не приймає жодних

даних, наявних у Din, і зберігає свій вихід Dout у так званому стані високого опору, який фактично від'єднує її від решти системи. На наростаючому фронті комірка або приймає стан Din як новий біт для збереження, або встановлює Dout на поточне збережене значення (рисунок 2.2).

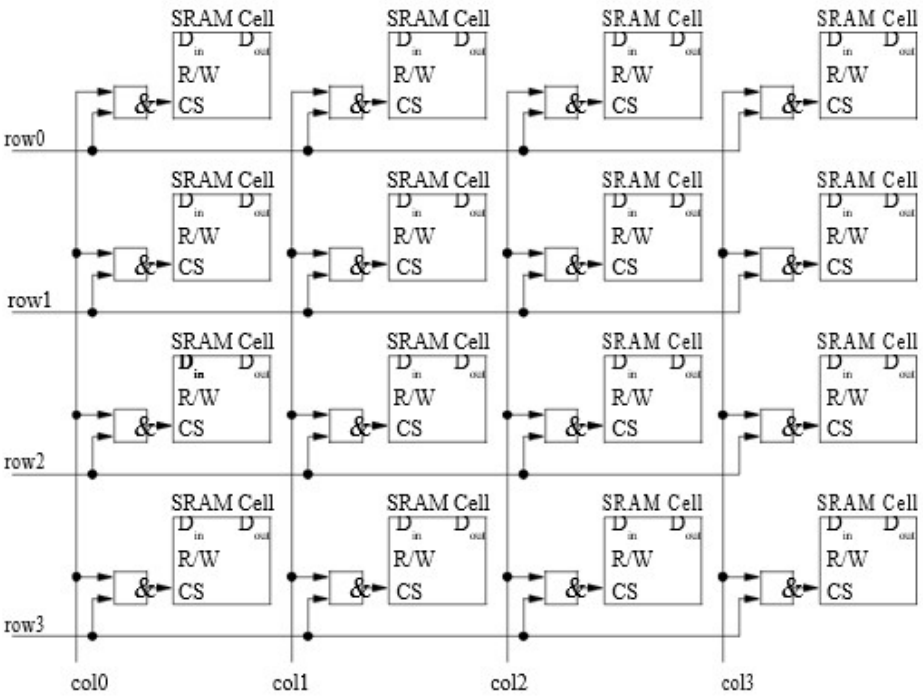


Рисунок 2.2 – Матриця пам'яті

Щоб отримати корисну пам'ять, багато таких комірок упорядковують у матриці, як показано на рисунку 2.2. Як бачите, усі лінії Даута пов'язані разом. Якщо всі комірки керуватимуть своїми виходами, незважаючи на відсутність адреси, може статися коротке замикання між GND і VCC, що, швидше за все, знищить мікросхему. Таким чином, лінія CS використовується для вибору однієї комірки в матриці та переведення всіх інших комірок у стан високого опору. Щоб адресувати одну клітинку і, отже, отримати доступ до одного конкретного біта, SRAM потребує додаткової логіки для полегшення такої адресації (зверніть увагу, що ми використовуємо, звичайно, спрощену схему).

Як ви можете бачити на рисунку 2.2, до певної комірки пам'яті звертаються (тобто її CS піднімається на високий рівень), коли пов'язані з нею рядок і стовпець піднімаються на високий рівень (маленькі квадрати з

амперсандом у них є і-воротами, вихід яких високий точно тоді, коли обидва входи є високими). Метою, звичайно, є збереження адресних рядків. Якби ми адресували кожну комірку за допомогою окремого рядка, наприклад, для оперативної пам’яті 16Kx1 (16K біт) уже знадобилося б 16384 рядки. Використовуючи матричну структуру з одним and-gate на комірку, достатньо 256 рядків[16].

Хоча 256 адресних рядків набагато краще, ніж 16384, це все ще неприйнятно багато для такого простого пристрою, як 16Kx1 RAM – така кількість контактів зробить ваш звичайний мікроконтролер блідим у порівнянні. Однак ми можемо ще більше зменшити кількість адресних рядків: у будь-який момент часу можна вибрати не більше одного рядка та одного стовпця – інакше ми звернемося до кількох комірок пам’яті одночасно. Ми можемо використовувати цей факт, щоб зменшити кількість необхідних рядків, додавши так звані декодери. n-розрядний декодер — це компонент із n вхідними контактами та 2n вихідними контактами, які пронумеровані від O0 до O2n−1. На вхідних виводах представлено двійкове число b, і декодер встановлює Ob в 1, а всі інші виходи в 0. Отже, замість фактичного встановлення одного з багатьох рядків, нам просто потрібен номер рядка, який ми хочемо вибрати, і декодер створює фактичні рядки рядків. З цією зміною наш 16Kx1 SRAM потребує не більше 14 рядків адреси.

На відміну від загальновідомого твердження, що нікому ніколи не знадобиться більше 640 кілобайт оперативної пам'яті, здається, що доступної пам'яті ніколи не буде достатньо. Очевидно, що ми хотіли б отримати якомога більший обсяг пам’яті з чіпа пам’яті певного розміру.

Відомо, що SRAM зазвичай потребує шести транзисторів для зберігання одного біта інформації. Звичайно, чим більше транзисторів на комірку потрібно, тим більшою буде площа кремнію. Якби ми змогли зменшити кількість необхідних компонентів – скажімо, ми використовуємо лише вдвічі менше транзисторів – тоді ми отримали б приблизно вдвічі більшу ємність накопичувача.

Саме цього вдалося досягти за допомогою динамічної пам’яті з довільним доступом (DRAM): кількість транзисторів, необхідних для кожного біта інформації, було зменшено до одного. Це, звичайно, зменшувало площу кремнію для заданої кількості клітин. Отже, при тому самому розмірі мікросхеми DRAM має набагато більшу ємність, ніж SRAM.

Замість того, щоб використовувати багато транзисторів для створення тригерів, один біт інформації зберігається в конденсаторі. Пам'ятаєте конденсатори? Вони працюють як маленькі акумуляторні батареї – ви прикладаєте до них напругу, і вони зберігають цю напругу. Відключіть, і у вас завантажений конденсатор. З’єднайте висновки навантаженого конденсатора через резистор, і потече електричний струм, який розрядить конденсатор.

А де той транзистор на комірку пам’яті, про який ми говорили, оскільки інформація зберігається в конденсаторі? Що ж, інформація справді зберігається в конденсаторі, але щоб вибрати її для читання чи запису, потрібен транзистор [17].

Тепер має бути зрозуміло, як працює DRAM: якщо ви хочете зберегти логічну пам’ять, ви звертаєтеся до комірки пам’яті, до якої хочете отримати доступ, керуючи транзистором. Потім ви прикладаєте напругу, яка заряджає конденсатор. Щоб зберегти логічний нуль, ви вибираєте комірку та розряджаєте конденсатор. Хочете прочитати свою інформацію? Ну, вам просто потрібно перевірити, заряджений конденсатор чи ні.

Щоб вирішити проблему оновлення, доступні DRAM із вбудованою логікою оновлення, але ця додаткова логіка займає значну частину мікросхеми, що є дещо контрпродуктивним і не обов’язково потрібним: часто ЦП не потребує доступу до своїх ОЗУ кожного циклу, але також має внутрішні цикли для виконання своєї фактичної роботи. Логіка контролера оновлення DRAM може використовувати цикли між зверненнями до ЦП для виконання оновлення.

DRAM має приблизно в чотири рази більшу ємність, ніж SRAM, приблизно за таку саму вартість і розмір мікросхеми. Це означає, що DRAM

доступні більшої ємності. Однак це також збільшило б кількість контактів адреси ⇒ більший пакет для їх розміщення ⇒ вищу вартість. Тому має сенс зменшити кількість зовнішніх контактів шляхом мультиплексування номерів рядків і стовпців: по-перше, номер рядка представлений на контактах адреси, які DRAM зберігає внутрішньо. Потім наводиться номер стовпця. DRAM поєднує його з попередньо збереженим номером рядка, щоб сформувати повну адресу.

Окрім необхідності оновлення пам’яті, DRAM має ще один серйозний недолік: він набагато повільніший, ніж SRAM. Однак через високу вартість SRAM це просто не варіант для звичайних настільних ПК. Тому було розроблено численні варіанти методів доступу до DRAM, що постійно збільшують швидкість пам’яті DRAM [18].

2.3 Процесорне ядро мікроконтролерів

Базова архітектура центрального процесора складається з шляху даних, який виконує інструкції, і блоку керування, який в основному повідомляє шляху даних, що робити. Центральний процесор є найважливішою частиною контролера, так як відповідає за роботу та виконання задач контролером, які надає користувач (рисунок 2.3).

В основі центрального процесора лежить арифметико-логічний пристрій (ALU), який використовується для виконання обчислень (AND, ADD, INC, . . .). Кілька рядків керування вибирають, яку операцію ALU має виконувати над вхідними даними.

ALU отримує два входи та повертає результат операції як вихід. Джерело та призначення беруться з реєстрів або з пам'яті.

Регістровий файл містить робочі регістри центрального процесора. Він може складатися або з набору регістрів загального призначення (зазвичай 16–32, але їх також може бути більше), кожен з яких може бути джерелом або одержувачем операції, або складається з кількох спеціальних регістрів.

Спеціальні регістри, напр. акумулятор, який використовується для арифметичних/логічних операцій, або індексний регістр, який використовується для деяких режимів адресації [19].

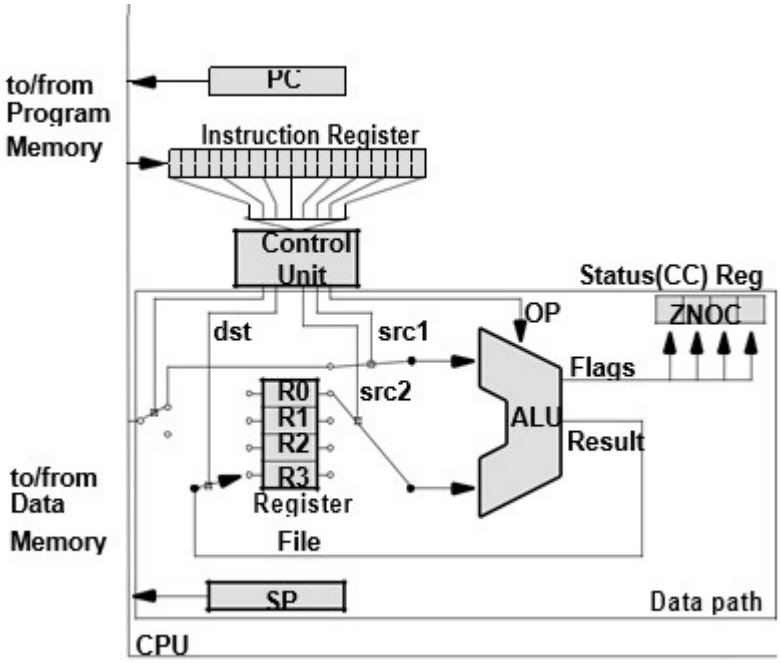


Рисунок 2.3 – Базова архітектура процесора

Крім того, ALU зберігає деяку інформацію про природу результату в регістрі стану (також називається регістром коду умови):

- Z (нуль): результат операції дорівнює нулю.
- N (Негативний): Результат операції негативний, тобто встановлюється старший біт (msb) результату (1).
- O (переповнення): операція призвела до переповнення, тобто відбулася зміна знака в операції доповнення до двох.
- C (перенесення): операція створила перенесення.

У будь-якому випадку ЦП може взяти операнди для ALU з файлу та зберегти результат операції назад у файл реєстру. Крім того, операнди/результат можуть надходити з пам’яті/зберігатися в пам’яті. Однак доступ до пам'яті набагато повільніший, ніж доступ до файлу реєстру, тому, як правило, доцільно використовувати файл реєстру, якщо це можливо.

Стек — це частина послідовної пам’яті в просторі даних, яка використовується центральним процесором для зберігання адрес повернення та, можливо, вмісту реєстрації під час викликів підпрограми та підпрограми обслуговування переривань. Доступ до нього здійснюється за допомогою команд PUSH (помістити щось у стек) і POP (вилучити щось зі стеку). Для збереження поточного рівня заповнення стека ЦП містить спеціальний регістр, званий покажчиком стека (SP), який вказує на вершину стека. Стеки зазвичай ростуть «вниз», тобто від вищих адрес пам’яті до нижчих адрес. Таким чином, SP зазвичай починається в кінці пам’яті даних і зменшується з кожним натисканням і збільшується з кожним випаданням. Причина розміщення вказівника стека в кінці пам’яті даних полягає в тому, що ваші змінні, як правило, знаходяться на початку пам’яті даних, тому, розміщуючи стек у кінці пам’яті, для їх зіткнення потрібно більше часу [20].

Окрім деяких особливих ситуацій, таких як інструкція HALT або скидання, ЦП постійно виконує програмні інструкції. Завдання блоку керування — визначити, яку операцію слід виконати наступною, і відповідно налаштувати шлях передачі даних. Для цього використовується інший спеціальний регістр, програмний лічильник (PC), для зберігання адреси наступної команди програми. Блок керування завантажує цю інструкцію в регістр інструкцій (IR), декодує інструкцію та встановлює шлях даних для її виконання. Конфігурація шляху даних включає надання відповідних входів для ALU (з регістрів або пам’яті), вибір правильної операції ALU та переконання, що результат записаний у правильне місце призначення (регістр або пам’ять). PC або збільшується, щоб вказати на наступну інструкцію в послідовності, або завантажується з новою адресою у випадку переходу або виклику підпрограми. Після скидання ПК зазвичай ініціалізується на $0000.

Традиційно блок керування був жорстко підключеним, тобто він в основному містив пошукову таблицю, яка містила значення контрольних ліній, необхідних для виконання інструкції, а також досить складну логіку декодування. Це означало, що було важко змінити або розширити набір

інструкцій ЦП. Щоб полегшити конструкцію блоку керування, Моріс Вілкс подумав, що блок керування фактично сам по собі є невеликим процесором і може отримати користь від власного набору мікроінструкцій. У його подальшому проектуванні блоку керування програмні інструкції були розбиті на мікроінструкції, кожна з яких виконувала деяку невелику частину всієї інструкції (наприклад, надання правильного регістру для ALU). Це, по суті, перетворило дизайн керування на завдання програмування: додавання нової інструкції до набору інструкцій зводилося до програмування інструкції в мікрокоді. Як наслідок, раптово стало відносно легко додавати нові та складні інструкції, а набори інструкцій стали досить великими та потужними. Це принесло назву архітектурі Complex Instruction Set Computer (CISC). Звичайно, потужний набір інструкцій має свою ціну, і цією ціною є швидкість: мікрокодовані інструкції виконуються повільніше, ніж жорсткі. Крім того, дослідження показали, що лише 20% інструкцій машини CISC відповідають за 80% коду (правило 80/20). Це та той факт, що ці складні інструкції можуть бути реалізовані за допомогою комбінації простих, викликали рух назад до простих жорстких архітектур, які були відповідно названі комп'ютером зі скороченим набором інструкцій (RISC).

Звичайно, коли у вас дві архітектури, виникає питання, яка з них краща. У випадку RISC проти CISC відповідь залежить від того, що вам потрібно. Якщо ваше рішення часто використовує потужні інструкції або режим адресації певної архітектури CISC, вам, ймовірно, буде краще використовувати CISC. Якщо вам потрібні головним чином прості інструкції та режими адресації, вам, швидше за все, краще використовувати RISC. Звичайно, цей вибір також залежить від інших факторів, таких як тактова частота відповідних процесорів. У будь-якому випадку, ви повинні знати, що вам потрібно від архітектури, щоб зробити правильний вибір.

Регістровий файл містить робочі регістри центрального процесора. Він може складатися або з набору регістрів загального призначення (зазвичай 16–32, але їх також може бути більше), кожен з яких може бути джерелом або

одержувачем операції, або складається з кількох спеціальних регістрів. Спеціальні регістри, напр. акумулятор, який використовується для арифметичних/логічних операцій, або індексний регістр, який використовується для деяких режимів адресації [19].

2.4 Переривання мікроконтролерів

Мікроконтролери, як правило, розгортаються в системах, які повинні реагувати на події. Події означають зміни стану в керованій системі і, як правило, вимагають певної реакції мікроконтролера. Реакції варіюються від простих відповідей, таких як збільшення лічильника щоразу, коли заготовка перетинає фотоелектричний бар'єр на конвеєрній стрічці, до критичних заходів, таких як вимкнення системи, якщо хтось потягнеться до робочої зони машини. Якщо припустити, що контролер може спостерігати за подією, тобто існує вхідний рядок, який змінює свій стан, щоб вказати подію, залишається питання про те, як контролер повинен контролювати вхідний рядок, щоб забезпечити правильну та своєчасну реакцію [21].

Звичайно, можна просто опитувати вхідний сигнал, тобто періодично перевіряти зміни стану. Однак це опитування має свої недоліки: воно не тільки зайво витрачає процесорний час, якщо подія відбувається лише рідко, його також важко змінити або розширити. Зрештою, мікроконтролеру, як правило, потрібно зробити набагато більше, ніж просто чекати на одну подію, тому подія періодично опитується таким чином, що решта програми також може бути виконана. З іншого боку, сигнал, можливо, доведеться опитувати з певним максимальним періодом, щоб уникнути пропущених подій, тому код опитування, можливо, доведеться викликати з кількох місць у основній програмі. Це вже займає багато часу, щоб визначити, з яких позицій у коді сигнал повинен бути опитаний в першу чергу, і ці позиції повинні бути переглянуті щоразу, коли основний код змінюється. Отже, опитування

незабаром втрачає свою привабливість, і розробник програмного забезпечення починає шукати інші способи обробки цих рідкісних подій.

На щастя, сам мікроконтролер пропонує зручний спосіб у вигляді переривань. Тут мікроконтролер опитує сигнал і перериває основну програму лише в разі виявлення зміни стану. Поки немає зміни стану, основна програма просто виконується без жодних побоювань щодо події. Як тільки відбувається подія, мікроконтролер викликає програму обслуговування переривань (ISR), яка обробляє подію. ISR повинен бути наданий прикладним програмістом.

Оскільки контролер має більше ніж одне джерело переривань і може фактично містити досить багато різних переривань, виникає питання, як розглядати ситуації, коли дві або більше події переривань відбуваються одночасно. Це не так малоймовірно, як ви могли б подумати, особливо якщо інколи програма вимикає переривання. Отже, повинна бути доступна детермінована та розумна стратегія для вибору наступного переривання для обслуговування.

Більшість контролерів із великою кількістю переривань і таблицею векторів використовують вектор переривань як індикацію пріоритету. Наприклад, ATmega16 статично призначає найвищий пріоритет перериванню з найменшим вектором переривання. Якщо контролер пропонує NMI, вони, швидше за все, матимуть найвищий пріоритет.

Але пріоритети можна використовувати не тільки для того, щоб визначити, хто виграє змагання, вони також можуть бути використані, щоб визначити, чи може переривання перервати активний ISR: у таких системах, якщо ввімкнено, переривання з вищим пріоритетом перерве ISR переривання з нижчим пріоритетом (вкладене переривання). Інші контролери, напр. ATmega16 дозволяє будь-якому перериванню переривати ISR, якщо встановлено біт дозволу переривання. Оскільки переривання не завжди бажане, багато контролерів вимикають глобальний біт IE перед виконанням ISR або надають інші засоби для ISR, щоб вибрати, чи можна його переривати чи ні.

Звичайно, статичне призначення пріоритетів не завжди може відображати вимоги прикладної програми. Як наслідок, деякі контролери дозволяють користувачеві динамічно призначати пріоритети принаймні деяким перериванням. Інші дозволяють користувачеві вибрати в ISR, яким перериванням слід дозволити переривати ISR.

2.5 Контролер STM32

Сімейство 32-розрядних флеш-мікроконтролерів STM32 базується на революційному ядрі ARM Cortex-M3 – ядрі, спеціально розробленому для вбудованих додатків, які вимагають поєднання високої продуктивності, роботи в режимі реального часу, низької потужності та недорогої роботи. Сімейство STM32 має переваги архітектури Cortex-M3 удосконалення (включаючи набір інструкцій Thumb-2), які забезпечують покращену продуктивність у поєднанні з кращою щільністю коду, і тісно зв'язаний векторизований контролер переривань, який значно прискорює реакцію на переривання, у поєднанні з провідним у галузі споживанням енергії. STMicroelectronics була провідним партнером у розробці ядра Cortex-M3 і зараз є першим провідним постачальником мікроконтролерів, який представив сімейство продуктів на основі ядра (рисунок 2.4).

Сімейство STM32 створено, щоб запропонувати нові ступені свободи користувачам мікроконтролерів. Він пропонує повний асортимент 32-розрядних продуктів, які поєднують високу продуктивність, роботу в режимі реального часу, низьку потужність і низьку напругу, зберігаючи повну інтеграцію та легкість розробки. Він полегшує перехід із 16-розрядного світу завдяки високому рівню інтеграції функцій, зручній у використанні архітектурі, можливостям низького енергоспоживання та економічній ефективності.

STM32 побудовано на 32-розрядній RISC-архітектурі промислового стандарту ARM. Cortex-M3 – найновіше ядро від ARM. Ядро Cortex M3

орієнтоване на ринки мікроконтролерів і вбудованих пристроїв. Удосконалені архітектурні особливості процесора Cortex-M3 зменшують розмір пам’яті, забезпечуючи найкращу в галузі продуктивність у невеликому енергоефективному ядрі RISC. Таким чином, він забезпечує ідеальну платформу для міграції багатьох різноманітних додатків у всьому світі від застарілих пристроїв до світу 32-розрядних мікроконтролерів.



Рисунок 2.4 – Чип STM32

У режимі очікування споживання струму становить лише 2 мкА, з активною схемою скидання. Нарешті, джерело живлення від 2,0 В до 3,6 В дозволяє використовувати його для додатків, що працюють від акумулятора. STM32 має три різних режими низького енергоспоживання та універсальну схему тактування, щоб користувачі могли оптимізувати енергоспоживання в порівнянні з продуктивністю. STM32 також містить годинник реального часу (RTC), який працює або від кварцового генератора 32 кГц, або від внутрішнього RC-генератора. RTC має окремий домен живлення з вбудованим перемикачем для роботи від спеціальної батареї типу «таблетка» або від основного джерела живлення. Його типове споживання струму становить 1,4 мкА при 3,3 В. Він вбудовано до 84 байтів для резервного копіювання даних. Час запуску з режимів низького споживання менше ніж 6 мкс, як правило, з режиму зупинки та 50 мкс, як правило, з режиму очікування та скидання (рисунок 2.5).

Оптимізована криптобібліотека для 32-розрядного мікроконтролера STM32, в основному для потреб безпеки даних. 128-розрядний алгоритм симетричного ключа AES-ECB у поєднанні з STM32 Cortex-M3 забезпечує чудову продуктивність із надійним захистом даних. 128-бітний тест AES-ECB для STM32 доступний за запитом із 16-, 32-, 64- та 128-байтним відкритим текстом для інструментальних ланцюжків Keil, IAR і Hitex.



Рисунок 2.5 – Discovery плата STM32

Бібліотека мікропрограм STM32 забезпечує легкий доступ до всіх функцій стандартних периферійних пристроїв STM32. Цей пакет безкоштовного програмного забезпечення містить драйвери для всіх стандартних функцій пристрою та периферійних пристроїв, від GPIO та таймерів до CAN, I2C, FSMC, I2S, SDIO, DAC, SPI, UART, ADC тощо. Повністю задокументований і протестований вихідний код C вимагає лише базових знань програмування на C, сумісний з будь-яким компілятором C для мікроконтролерів на основі ядра ARM і відповідає MISRA C. Бібліотека STM32 має ті самі API, що й бібліотеки STR7 і STR9.

Оптимізовані та задокументовані бібліотеки мікропрограм C для керування як PMSM, так і асинхронними двигунами змінного струму у векторному режимі (FOC) надаються безкоштовно за запитом. Ці модульні бібліотеки підтримують обидва типи двигунів в автономному режимі за допомогою обладнання ST. Вихідні файли надаються безкоштовно та відповідають вимогам стандарту MISRA C (останні правила), що сприяє відповідності IEC 60730.

3 ЗАСОБИ РОЗРОБКИ

При розробці програмного продукту важливим чинником є правильний вибір засобів програмної реалізації, що впливає на час розробки, якість, надійність та швидкість кінцевого продукту. Також важливо враховувати особливості, можливості й поширення операційної системи, під керівництвом якої буде виконуватися програма.

3.1 Мова програмування C++

Раніше програми писалися мовою машин або ж машинним кодом, який складався з примітивних інструкцій які могли були прочитані лише машиною. Програми написані машинним кодом були важкими для розуміння, зазвичай тому що структура машинного коду відбивала дизайн комп'ютера а ніж програміста. Гірше було навіть те що кожна машина мала свій дизайн та код який тільки вона розуміє [22].

В середині 50-тих років, група програмістів під керівництвом Джона Бакуса в IBM мали ідею, яка повністю змінила бачення та природу комп'ютерів. Бакус та його колеги думали а чи можна писати чисто математичний код та формули і нехай сам комп'ютер перекладає його у машинний код. В 1955 команда створила першу версію Smalltalk, який був першим прикладом високорівневої мови програмування.

З того часу багато мов було створено більшість з яких перевершували минулі. C++ також була в цій гілці еволюції. Одним з її предків була мова C, яка була створена Bell Laboratories Денісом Рітше в 1972 році, а далі стандартизована American National Standard Institute в 1989 [23].

Не зважаючи на багато привабливих речей і високу інтерактивність середовища користувача яка спрощує процес програмування Smalltalk ніколи не був комерційно успішним. Професіонали проявляли свій інтерес до об'єктно-орієнтованого програмування тільки після того як центральні ідеї

були були імплементовані в варіаціїї С, які які вже стали стандартом індустрії. Але було кілька паралельних спроб створити об'єктно-орієнтовану мову на базі С, найбільш успішним став С++, який створив Бйорне Строустрап в Bell Laboratories в ранні 80-ті. С++ включав в себе стандарти з мови С, які давали змогу легко інтегрувати С++ код в уже існуючі С програми дуже легко [24].

Але сказати що об'єктно-орієнтовані мови програмування отримали свою популярність за рахунок зменшення популярності процедурних було б помилкою. Програмні парадигми не такі вже й конкуруючі настільки наскільки вони комплементарні. Об'єктно-орієнтована парадигма та процедурна на ряду з іншими важливими парадигмами, такими як функціональне програмування в LISP всі мають важливе практичне призначення.

Процес компіляції в С++. Коли ви пишете програму на С++, ваший перший крок це створення файлу, який має містити текст програми, який називається source файл. Після того ви можете запустити вашу програму в якій проходить процес перекладу source файлу в скомпілюовану форму. Перший крок в цьому процесі запустити програму, яка називається компілятор, яка у свою чергу перекладає source файл в object файл, який містить у собі машинний код та інструкції. Цей файл потім комбінується з іншими файлами щоб створити відтворюваний файл, який може бути запущений в системі. Інші object файли, як правило раніше створені називаються бібліотеками, які мають машинний код для різних операцій, які потрібні програмі. Цей процес комбінації всіх індивідуальних об'єктів називається поєднання [25].

Навіть у примітивної програми HelloWorld, є специфічні деталі процесу компіляції відрізняються від машини до машини. Ні одна книжка не скаже вам що вам потрібно щоб саме ця програма спрацювала на вашій машині. Гарні новини це те що програми написані на С++ будуть виглядати однаково. Одна з переваг роботи з мовою програмування високого рівня такою як С++ це те що програмування дає змогу вам ігнорувати характеристики машини та створювати програми які будуть працювати на безлічі комп'ютерів.

3.2 Платформа на базі Arduino

Проєкт Arduino – це компанія з відкритим кодом апаратного та програмного забезпечення, проєкти та спільнота користувачів, яка розробляє та виробляє одноплатні мікроконтролери та комплекти мікроконтролерів для створення цифрових пристроїв. Його апаратні продукти ліцензуються згідно з ліцензією CC BY-SA, тоді як програмне забезпечення ліцензується відповідно до GNU Lesser General Public License (LGPL) або GNU General Public License (GPL), що дозволяє будь-кому виготовляти плати Arduino та розповсюджувати програмне забезпечення. Плати Arduino доступні на офіційному веб-сайті або через авторизованих дистриб'юторів [26].

Конструкції плат Arduino використовують різні мікропроцесори та контролери. Плати оснащені наборами контактів цифрового та аналогового входу/виводу (I/O), які можна підключати до різних плат розширення («щитів») або макетних плат (для створення прототипів) та інших схем. Плати мають інтерфейси послідовного зв'язку, включаючи універсальну послідовну шину (USB) на деяких моделях, які також використовуються для завантаження програм. Мікроконтролери можна програмувати за допомогою мов програмування C і C++, використовуючи стандартний API, який також відомий як мова Arduino Ino, натхненний мовою обробки та використовується з модифікованою версією IDE обробки. На додаток до використання традиційних інструментів компілятора, проєкт Arduino надає інтегроване середовище розробки (IDE) і інструмент командного рядка, розроблений у Go.

Проєкт Arduino розпочався в 2005 році як інструмент для студентів Інституту проєктування взаємодії Івреа, Італія, з метою забезпечити недорогий і простий спосіб для новачків і професіоналів створювати пристрої, які взаємодіють з навколишнім середовищем за допомогою датчиків і приводів. Типові приклади таких пристроїв, призначених для любителів-початківців, включають прості роботи, термостати та детектори руху.

Назва Arduino походить від бару в Івреа, Італія, де зустрічалися деякі із засновників проєкту. Бар був названий на честь Ардуїна з Івреї, який був маркграфом марки Іврея та королем Італії з 1002 по 1014 рік [27].

3.3 Платформа на базі STM32

Сімейство 32-розрядних флеш-мікроконтролерів STM32 базується на революційному ядрі ARM Cortex-M3 – ядрі, спеціально розробленому для вбудованих додатків, які вимагають поєднання високої продуктивності, роботи в режимі реального часу, низької потужності та недорогої роботи. Сімейство STM32 має переваги архітектури Cortex-M3 удосконалення (включаючи набір інструкцій Thumb-2), які забезпечують покращену продуктивність у поєднанні з кращою щільністю коду, і тісно зв'язаний векторизований контролер переривань, який значно прискорює реакцію на переривання, у поєднанні з провідним у галузі споживанням енергії. STMicroelectronics була провідним партнером у розробці ядра Cortex-M3 і зараз є першим провідним постачальником мікроконтролерів, який представив сімейство продуктів на основі ядра [28].

Оптимізована криптобібліотека для 32-розрядного мікроконтролера STM32, в основному для потреб безпеки даних. 128-розрядний алгоритм симетричного ключа AES-ECB у поєднанні з STM32 Cortex-M3 забезпечує чудову продуктивність із надійним захистом даних. 128-бітний тест AES-ECB для STM32 доступний за запитом із 16-, 32-, 64- та 128-байтним відкритим текстом для інструментальних ланцюжків Keil, IAR і Hitex.

STM32 побудовано на 32-розрядній RISC-архітектурі промислового стандарту ARM. Cortex-M3 – найновіше ядро від ARM. Ядро Cortex M3 орієнтоване на ринки мікроконтролерів і вбудованих пристроїв. Удосконалені архітектурні особливості процесора Cortex-M3 зменшують розмір пам'яті, забезпечуючи найкращу в галузі продуктивність у невеликому енергоефективному ядрі RISC. Таким чином, він забезпечує ідеальну

платформу для міграції багатьох різноманітних додатків у всьому світі від застарілих пристроїв до світу 32-розрядних мікроконтролерів.

3.4 Середовище розробки Arduino IDE

Середовище розробки Arduino складається з вбудованого текстового редактора програмного коду, області повідомлень, вікна виведення тексту (консолі), панелі інструментів з кнопками команд, що часто використовуються, і кількох меню. Для завантаження програм та зв'язку середовище розробки підключається до апаратної частини Arduino [29].

Програма, написана серед Arduino, називається скетч. Скетч пишеться в текстовому редакторі, який має інструменти вирізки/вставки, пошуку/заміни тексту. Під час збереження та експорту проекту в області повідомлень з'являються пояснення, також можуть відображатися помилки. Вікно виведення тексту (консоль) показує повідомлення Arduino, які містять повні звіти про помилки та іншу інформацію. Кнопки панелі інструментів дозволяють перевірити та записати програму, створити, відкрити та зберегти скетч, відкрити моніторинг послідовної шини:

Середовищем Arduino використовується принцип блокнота: стандартне місце для зберігання програм (скетчів). Скетчі із блокнота відкриваються через меню File > Sketchbook або кнопкою Open на панелі інструментів. При першому запуску програми Arduino автоматично створюється директорія блокнота. Розташування блокнота змінюється через діалогове вікно Preferences [30].

Бібліотеки додають додаткову функціональність скетчам, наприклад, при роботі з апаратною частиною або обробкою даних. Щоб використовувати бібліотеку, виберіть меню Sketch > Import Library. Одна або кілька директив #include будуть розміщені на початку коду скетчу з подальшою компіляцією бібліотек та разом зі скетчем. Завантаження бібліотеки потребує додаткового

місця в пам'яті Arduino. Невикористані бібліотеки можна видалити зі скетчу, прибравши директиву #include.

Arduino.cc містить список бібліотек. Деякі бібліотеки входять у середу розробки Arduino. Інші можуть бути завантажені з різних ресурсів. Для встановлення завантажених бібліотек необхідно створити директорію «libraries» у папці блокнота та розпакувати архів. Наприклад, для встановлення бібліотеки DateTime її файли повинні знаходитись у підпапці /libraries/DateTime папки блокнота.

Деякі параметри змінюються у вікні Preferences (меню Arduino в Mac або File в Windows і Linux). Інші налаштування знаходяться у файлі, розташування якого вказано у вікні Preferences.

4 ОПИС ПРОГРАМНОЇ РЕАЛІЗАЦІЇ

В цьому розділі приведений опис програмної реалізації адаптивної системи управління автономним роботом.

4.1 Діаграма варіантів використання

Діаграма варіантів використання – це динамічна або поведінкова діаграма в UML. Діаграми варіантів використання моделюють функціональність системи за допомогою акторів і варіантів використання. Варіанти використання – це набір дій, послуг і функцій, які система повинна виконувати. У цьому контексті «система» – це щось, що розробляється або використовується, наприклад веб-сайт. «Актори» – це люди або організації, які виконують певні ролі в системі.

Діаграма прецедентів для системи зображено на рис. 4.1

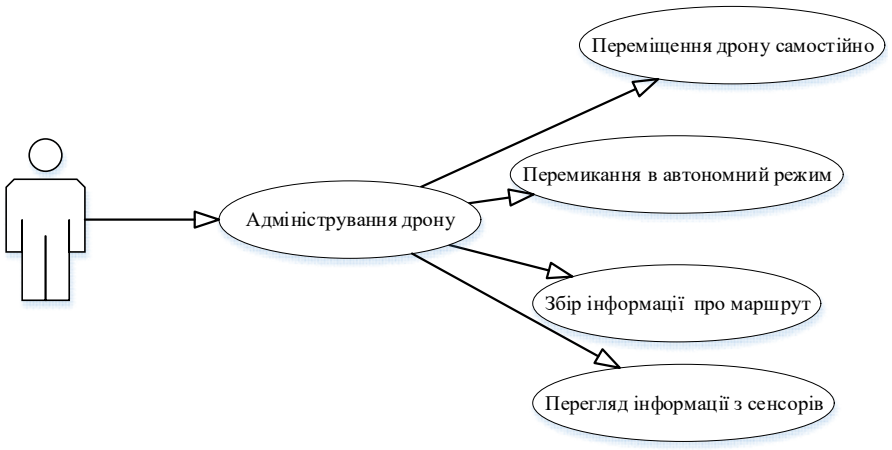


Рисунок 4.1 – Діаграма прецедентів

Діаграми варіантів використання є цінними для візуалізації функціональних вимог системи, які перетворяться на вибір дизайну та

пріоритети розробки. Вони також допомагають визначити будь-які внутрішні чи зовнішні фактори, які можуть впливати на систему та які слід враховувати.

Вони забезпечують хороший аналіз високого рівня ззовні системи. Діаграми варіантів використання визначають, як система взаємодіє з акторами, не турбуючись про деталі того, як ця функція реалізована.

4.2 Електронна схема

Електронна схема складається з окремих електронних компонентів, таких як резистори, транзистори, конденсатори, котушки індуктивності та діоди, з'єднані провідними проводами або ланцюгами, через які може протікати електричний струм. Це тип електричного кола, і щоб його називали електронним, а не електричним, зазвичай має бути присутнім принаймні один активний компонент. Комбінація компонентів і проводів дозволяє виконувати різноманітні прості та складні операції: сигнали можна посилювати, виконувати обчислення та переміщувати дані з одного місця в інше.

Електронна схема для системи зображена на рис. 4.2

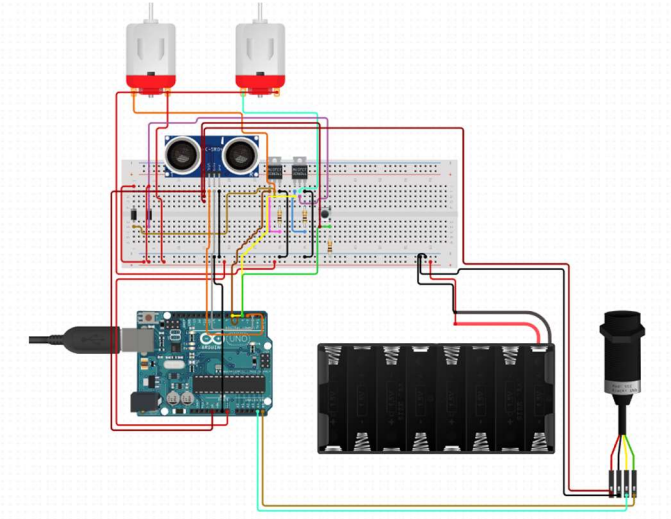


Рисунок 4.2 – Електронна схема

Дана схема демонструє взаємодію елементів робота між собою, та показує зв'язки деталей всередині машини.

Схеми можуть бути побудовані з окремих компонентів, з'єднаних окремими шматками дроту, але сьогодні набагато більш поширеним є створення з'єднань за допомогою фотолітографічних методів на ламінованій підкладці (друкованій платі чи друкованій платі) і припаювання компонентів до цих з'єднань для створення готової схема. В інтегральній схемі або IC компоненти та з'єднання сформовані на одній підкладці, як правило, на напівпровіднику, такому як легований кремній або (рідше) арсенід галію.

Для тестування нових дизайнів зазвичай використовують макетні панелі, перфоровані панелі та стрічкові панелі. Вони дозволяють розробнику швидко вносити зміни в схему під час розробки.

4.3 Структурна схема

Роботу системи можна розділити на 2 частини, а саме Front-end, те що бачить користувач, та Back-end, яка відповідає за те як процеси в системі відбуваються (рисунок 4.3).

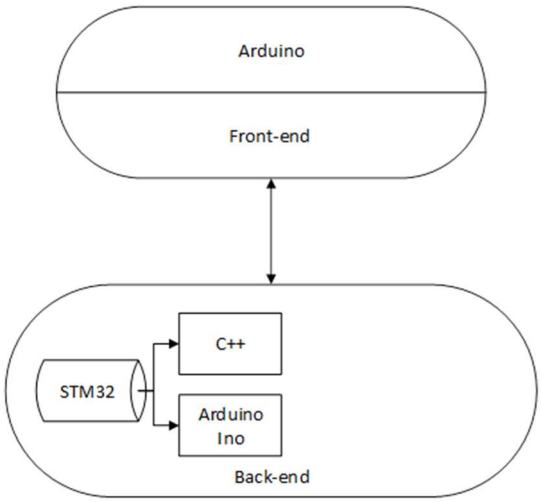


Рисунок 4.3 – Структура системи

Для розробки системи управління роботом була використана мова програмування C++ та Arduino Ino. Для проєктування робота був використаний контролер STM32 та конструктор Arduino.

4.4 Опис програмного коду

Перейдемо до опису коду який відповідає за роботу системи управління робота. Спочатку підключаємо бібліотеку для роботи з інфрачервоним пультом керування, та ініціалізуємо піни та змінні для кожного модулю робота (рисунок 4.4).

```
#include "IRremote.h"


int trigPin = 10;      // trig pin of HC-SR04
int echoPin = 11;     // echo pin of HC-SR04
long duration, distance;

IRrecv irrecv(6);
int motor1Pin1 = 3;
int motor1Pin2 = 4;

int motor2Pin1=7;
int motor2Pin2=8;
decode_results results;
```

Рисунок 4.4 – Ініцалізація пінів

Як бачимо з рисунку спочатку іде підключення бібліотеки а потім опис кожного піну, а саме 10 та 11 піни відповідають за роботу датчика звукової хвилі, 6 за інфрачервоний передавач, 3 та 4 за роботу першого мотору, 7 та 8 за роботу другого мотору (для роботи кожного мотору потрібні два піни, перший відповідає за рух вперед, другий за рух назад). А також проводимо опис змінних, duration та distance потрібні для обрахунку шляху робота при проходженні дистанції самостійно, results потрібна для роботи з інфрачервоним пультом, щоб витягувати числові значення для кнопок управління.

Далі переходимо до функції Setup, у якій проходить опис кожного піна на платі і в якому форматі він буде працювати. Як бачимо з рисунку кожен пін потребує окремого опису та вибору режиму роботи. Піни кожного з моторів ініціалізуються як OUTPUT, тому що рух оберти колесами це вихідні дані. Також ми вмикаємо інфрачервоний передавач та піни для роботи датчика звукової хвилі, а саме TriggerPin також працює в режимі OUTPUT, а EchoPin

буде працювати в режимі INPUT тому що буде передавати дані для обчислення відстані (рисунок 4.5).

```
void setup()
{

  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  pinMode(motor2Pin1, OUTPUT);
  pinMode(motor2Pin2, OUTPUT);
  Serial.begin(9600);
  irrecv.enableIRIn();
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);

}
```

Рисунок 4.5 – Функція Setup

Наступним кроком є функція loop в якій описуємо роботу машини. Функція починається з умови в якій ми дізнаємося чи натиснув користувач якусь кнопку. Далі за допомогою функції value витягаємо цифрові дані з results та в середині функції switch визначаємо яка саме кнопка була натиснута користувачем (рисунок 4.6).

```
void loop()
{
  if(irrecv.decode(&results))//&results
  {
    switch(results.value)
    {
```

Рисунок 4.6 – Функція Setup

Далі після отримання числового значення кнопки програма аналізує чи співпадає вона хоча б з одним значення у програмі. Якщо збіг знайдено йде перехід до іншої фази, а саме роботи з певною кнопкою.

Розглянемо кнопку для руху прямо. Після того як числове значення збіглося переходимо до роботи з двигунами. Перший пін кожного мотору ставимо у стан HIGH, а другі піни на LOW. HIGH та LOW означають чи працює на даний момент, пін при HIGH пін вмикається, при LOW

вимикається. Далі після отримання команди колеса робота починають обертатися вперед і дрон починає переміщення (рисунок 4.7).

```
case 424416959://move forward
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);
  digitalWrite(motor2Pin1, HIGH);
  digitalWrite(motor2Pin2, LOW);
break;
```

Рисунок 4.7 – Функція для їзди прямо

Далі розглянемо функцію для повороту ліворуч. Для того щоб повернути роботу треба провести такі дії як рух правим мотором прямо, а лівим мотором слід обертати колесо назад. Саме та робот зможе повернути ліворуч (рисунок 4.8).

```
case 788533224: // turn left
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);
  digitalWrite(motor2Pin1, LOW);
  digitalWrite(motor2Pin2, HIGH);
break;
```

Рисунок 4.8 – Функція для повороту ліворуч

Розглянемо функцію автоматично переміщення та об’їзду перешкод. Після того як числове значення кнопки збіглося з тим що є у програмі переходимо до обчислення дистанції між роботом та перешкодою. Для початку описуємо безкінечний цикл, вимикаємо TriggerPin на випадок якщо той увімкнений після затримки в дві мікросекунди пін вмикається та збирає інформацію посилаючи звукову хвилю, яка відбивається від перешкод та повертається назад у датчик. Потім за допомогою EchoPin зчитуємо як довго хвиля була у просторі та визначаємо відстань до перешкоди. Щоб визначити відстань нам потрібно час хвилі помножити на швидкість звуку та поділити на

2 так як у часі хвилі описується весь час життя хвилі у просторі, а саме вихід з датчика відбиття та повернення назад (рисунок 4.9).

```
case 400113176: // obastacle avoiding

  while(true)
  {
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    duration = pulseIn(echoPin, HIGH);
    distance = duration * 0.034 / 2;
    delayMicroseconds(10);
```

Рисунок 4.9 – Обчислення шляху проходження

Після того як дистанція була обчислена, за допомогою умов перевіряємо наскільки дистанція велика. Якщо відстань більша за 20 сантиметрів, то робот спокійно рухається прямо за допомогою функції яку описали вище (рисунок 4.10).

```
if (distance > 20) // condition for absence of obstacle
{
  digitalWrite(motor1Pin1, HIGH);//move forward
  digitalWrite(motor1Pin2, LOW);
  digitalWrite(motor2Pin1, HIGH);
  digitalWrite(motor2Pin2, LOW);
}
```

Рисунок 4.10 – Обчислення шляху проходження

Але якщо відстань більша за 20 сантиметрів використовується інша функція за допомогою якої робот зможе об’їхати перешкоду. А саме робот зупиниться та поїде назад знову зупиниться та після не великої затримки поверне ліворуч (рисунок 4.11).

```
if (distance < 20) // condition for presence of obstacle
{
  digitalWrite(motor1Pin1, LOW);//stop
  digitalWrite(motor1Pin2, LOW);
  digitalWrite(motor2Pin1, LOW);
  digitalWrite(motor2Pin2, LOW);
  delay(500);
  digitalWrite(motor1Pin1, LOW);//move backward
  digitalWrite(motor1Pin2, HIGH);
  digitalWrite(motor2Pin1, LOW);
  digitalWrite(motor2Pin2, HIGH);
  delay(500);
  digitalWrite(motor1Pin1, LOW);//stop
  digitalWrite(motor1Pin2, LOW);
  digitalWrite(motor2Pin1, LOW);
  digitalWrite(motor2Pin2, LOW);
  delay(100);
  digitalWrite(motor1Pin1, HIGH);//turn left
  digitalWrite(motor1Pin2, LOW);
  digitalWrite(motor2Pin1, LOW);
  digitalWrite(motor2Pin2, HIGH;
  delay(500);
}
if(irrecv.decode(&results))//&results
{
  break;
}
```

Рисунок 4.11 – Функція об’їзду перешкод

Як бачимо з рисунку спочатку функція перевіряє чи відстань не більша за 20 см. Далі передає у кожний пін кожного мотору LOW щоб зупинити робота після затримки робот від’їжджає назад, а саме програма передає у кожен другий пін моторів HIGH. Після чого робот знову зупиняється і після невеликої затримки повертає ліворуч. А потім цикл повторюється поки не буде виконана умова натискання якоїсь іншої кнопки.

5 РОБОТА КОРИСТУВАЧА З СИСТЕМОЮ

В цьому розділі приведені системні вимоги для роботи з системою та сценарії роботи користувача з нею.

5.1 Опис системних вимог

Так як система являє собою автономного робота для його роботи не потрібні ніякі системні вимоги, тому що ніякий додаток встановлювати не потрібно. Але якщо користувач захоче змінити якусь конфігурацію машини, то йому обов'язково знадобиться комп'ютер з портом USB 2.0 або вище. Також для роботи з кодом знадобиться Arduino IDE та C++ встановлені на даний комп'ютер, та всі можливі драйвери Arduino USB, які є у стандартному пакеті Arduino IDE.

5.2 Робота користувача з продуктом

Зовнішній вигляд робота у повному стані представлено на рисунку 5.1.

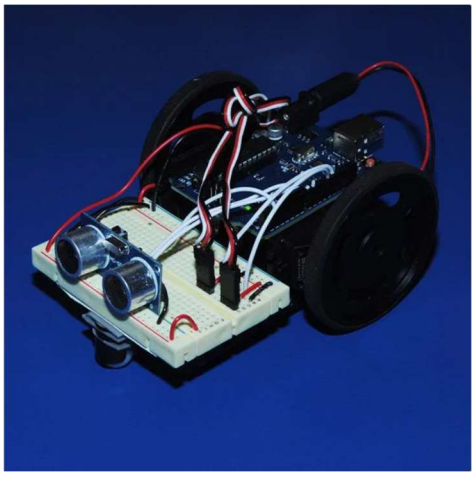


Рисунок 5.1 – Робот у повністю зібраному стані

Для початку роботи з роботом йог слід увімкнути. Зробити це можна за допомогою кнопки вбудованої в корпус яка знаходиться в передній частині

машини та допомагає користувачу розпочати взаємодію з системою машини (рисунок 5.2).

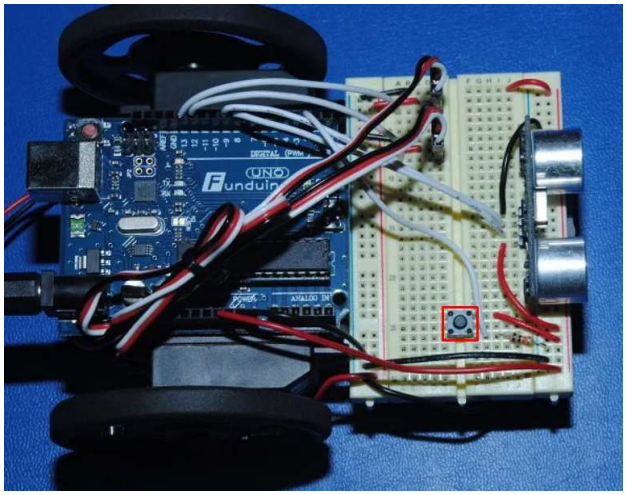


Рисунок 5.2 – Кнопка живлення

Також робота можна увімкнути з пульта керування. Сценарії якого розглянемо далі. Для керування роботом користувачу надається пульт. При натисканні кнопки «+» робот починає рухатися вперед (рисунок 5.3).



Рисунок 5.3 – Пульт для роботи з машиною

Розглянемо всі сценарії роботи які надає пульт. При натисканні кнопки «стрілка вправо» робот починає повертати праворуч тим самим рухати вперед починає лівий мотор а назад правий . При натисканні кнопки «-» робот спочатку зупиняється, а потім починає їхати назад. При натисканні кнопки

«стрілка вліво» робот починає повертати ліворуч тим самим рухати вперед починає правий мотор а назад лівий. При натисканні кнопки в центрі робот переходить в автономний режим в якому він може рухатися прямо самостійно, та обходити перешкоди. Для закінчення роботи з машиною треба знову натиснути кнопку живлення.

Користувач також в спромозі сам змінювати та модифікувати машину та її програмний код для взаємодії з пультом. Платформа Arduino надає можливість легко змінювати та покращувати робота. Можна легко додавати нові модулі, наприклад модуль камери (рисунок 5.4).



Рисунок 5.4 – Модуль камери

Щоб модуль працював достатньо лише додати цей невеликий код у програму управління де просто змінивши ID та пароль можна буде з легкістю користуватися камерою підключившись до неї зі смартфону (рисунок 5.5).

```
#define CAMERA_MODEL_M5STACK_ESP32CAM

const char* ssid = "******";

const char* password = "*******";
```

Рисунок 5.5 – Код для модулю камери

Також можна легко змінити модуль управління. Наприклад перенести його на смартфон за допомогою Bluetooth модуля, який допоможе легко

перевести управління робота на телефон длябільш зручної роботи та не використання пульта (рисунок 5.6).

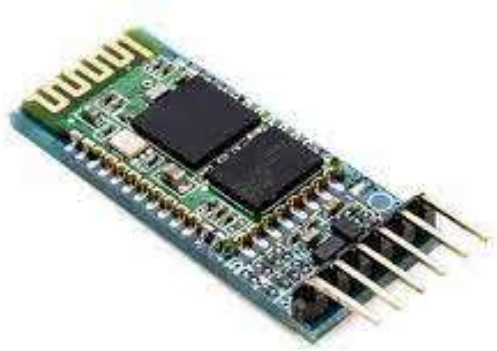


Рисунок 5.6 – Модуль Bluetooth

Щоб модуль працював достатньо лише додати цей невеликий код у програму управління а саме в функцію loop в коді і після цього скачавши додаток на смартфон можна з легкістю користуватися роботом зі смартфона (рисунок 5.7).

```
while(Serial.available()>0){
  inputByte= Serial.read();
  Serial.println(inputByte);
  if (inputByte=='Z'){
  digitalWrite(13,HIGH);
  }
  else if (inputByte=='z'){
  digitalWrite(13,LOW);
  }
  }
```

Рисунок 5.7 – Код для модулю Bluetooth

Можна додати модуль для роботи зі звуком. Щоб при знаходженні перешкоди або навіть просто при повороті робот видавав якийсь писк або

інший звук. У цьому нам допоможе модуль звуку, який можна з легкістю встановити на машину (рисунок 5.8).



Рисунок 5.8 – Модуль з динаміком

Достатньо додати цей у програму а саме у функцію loop, і робот уже почне видавати звуки (рисунок 5.9).

```
int val;
val=analogRead(0);   //connect mic sensor to Analog 0
Serial.print("Sound=");
Serial.println(val,DEC);//print the sound value to serial
lcd.setCursor(0, 0);
lcd.print("Sound=");
lcd.print(val);
lcd.print("    ");
delay(1000);
```

Рисунок 5.9 – Код для динаміку

Можна легко вставити цей код для якоїсь функціональної кнопки щоб робот міг відтворювати звук під час натискання на саме цю функціональну кнопку за допомогою функції switch case.

6. РОЗРОБЛЕННЯ СТАРТАП ПРОЄКТУ

У даному розділі було проведено аналіз та розробку стартап проєкту адаптивної системи управління автономним роботом.

6.1 Опис ідеї проєкту

Розпочнемо з переліку можливих варіантів використання продукту стартапу. Деталі описані в таблиці 6.1.

Таблиця 6.1

Опис ідеї стартапу

Зміст ідеї	Напрям застосування	Вигоди для користувача
Створення адаптивної системи управління автономним роботом	У військовій розвідці	Ефективне використання машини. Зменшення ризику людським життям.
	Використання машини для пошуку предметів	Зручне використання робота для пошуку загублених або захованих предметів.
	У поліції	Використання дрону для поліцейської розвідки, пошуку заручників та слідчих доказів.

Потім потрібно проаналізувати такі пункти:

- Визначення кількості технічних та економічних характеристик

- Визначити початковий перелік конкурентів
- Провести аналіз для порівняння показників

Аналіз описаний в таблиці 6.2.

Таблиця 6.2

Визначення сильних, слабких та нейтральних характеристик

№	Техніко-економічні характеристики ідеї	Товари/концепції конкурентів			W (слабка)	N (нейтральна)	S (сильна)
		Мій проєкт	Throwbot 2	Cobra MK2			
1.	Висока швидкість руху	Наявна	Наявна	Відсутня		N	
2.	Зручне управління системою	Наявна	Наявна	Наявна			S
3.	Малий розмір	Наявна	Наявна	Відсутня			S
4.	Можливість автономного керування	Наявна	Відсутня	Відсутня		N	
5.	Міцна конструкція	Наявна	Наявна	Наявна			S

6.2 Технологічний аудит ідеї проєкту

Щоб ефективно проробити ідею адаптивної системи управління автономним роботом, треба провести технологічний аудит. Визначення технологічної здійсненності ідеї проєкту та аналіз таких складових передбачає технологічний аудит за якою технологією буде виготовлено товар, чи існують

такі технології, чи доступні такі технології авторам проєкту. Опис аудиту наведений в таблиці 6.3.

Таблиця 6.3

Технологічна здійсненність проєкту

№	Ідея проєкту	Технології та її реалізації	Наявність технології	Доступність технологій
1.	Адаптивна система управління автономним роботом	C++	Наявна	У відкритому доступі
2.		Arduino ino	Наявна	У відкритому доступі
3.		Конструктор Arduino	Наявна	У відкритому доступі
4.		Чип з контролером STM32	Наявна	У відкритому доступі

Після перевірки результатів технологічного аудиту, можна зрозуміти що всі потрібні технології у вільному доступі.

6.3 Аналіз ринкових можливостей запуску стартап-проєкту

Далі потрібно провести аналіз ринку. Результати наведені в таблиці 6.4.

Таблиця 6.4

Попередня характеристика потенційного ринку стартапу

№	Показники стану ринку	Характеристика
1.	Кількість головних гравців, од	4-6

Продовження таблиці 6.4

2.	Загальний обсяг продаж, грн/ум.од	Середня вартість дрону для розвідки 20000$.
3.	Динаміка ринку	Швидко зростає
4.	Наявність обмежень для входу	Отримання військових сертифікацій.
5.	Специфічні вимоги до стандартизації ти сертифікації	Необхідні спеціальні військові сертифікати.

Наведемо ще таблицю, і в ній приведемо групи потенційних користувачів, їх вимоги та потреби. Наведено у таблиці 6.5.

Таблиця 6.5

Характеристика потенційних клієнтів

№	Потреба, що формує ринок	Цільова аудиторія	Відмінності у поведінці різних потенційних цільових груп	Вимоги споживачі до товару
1.	Потреба у простому управлінні	Поліцейські та військові	Високі вимоги до легкості управління	Гарантія надійної роботи системи, зрозуміла документація
2.	Висока автономність	Військові	Високі вимоги до довгої роботи машини	Гарантія надійної роботи системи,

Продовження таблиці 6.5

3.	Малий розмір	Звичайні юзери	Високі вимоги до мініатюрності дрону	Гарантія надійної роботи системи,

Після того як визначилися потенційні групи клієнтів потрібно провести аналіз ринку на фактори загроз(таблиця 6.6) та можливостей(таблиця 6.7).

Таблиця 6.6

Фактори загроз

№	Фактор	Зміст загрози	Можлива реакція
1.	Людський фактор	Неможливість прорахувати людський фактор у процесі роботи	Покращення системи
2.	Обмеженість функцій	Система обмежена її функціями, та не має деяких речей які є у конкурентів	Додавання нових функцій
3.	Висока частота помилок	Неправильне налаштування системи	Нові часті оновлення системи

Таблиця 6.7

Фактори можливостей

№	Фактор	Зміст можливості	Можлива реакція

Продовження таблиці 6.7

1.	Нові інвестиції	Можливість отримання інвестицій від інших компаній	Розширення штату, найм кращих спеціалістів
2.	Збільшення бази клієнтів	За допомогою реклами розширити клієнтську базу	Продумати рекламну кампанію, проводити заходи з демонстрацією продукту
3.	Слабкі конкуренти	Відсутність великої конкуренції	Завищення цін на продукт

Далі проводимо аналіз конкуренції на ринку, наведено у таблиці 6.8. Аналіз конкуренції є досить важливим фактором для стартапів.

Таблиця 6.8

Ступеневий аналіз ринку конкуренції

Особливості конкурентного середовища	В чому проявляється дана характеристика	Вплив на діяльність підприємства(можливі дії компанії щоб бути конкурентоспроможною)
1. Тип конкуренції - чиста	Велика кількість гравців на ринку які ні від кого не залежать	Покращити надійність системи
2. За рівнем конкурентної боротьби - міжнародний	Компанії виходять на міжнародний ринок	Потрібно працювати на світовому ринку

Продовження таблиці 6.8

3.За галузевою ознакою - глобальна	Продукт можна використовувати у різних	Покращення системи для різних галузей
4. Конкуренція видами товарів – товарно-видова	Конкуренція між продуктами та їх особливостями	Покращити продукт, за допомогою помилок конкурентів
5.За характером конкурентних переваг - цінова	Велика різниця в цінах різних постачальників	Обрати конкурентну ціну для системи
6. За інтенсивністю - марочна	Компанії виступають під своїми брендами	Створити бренд який легко впізнається

Далі проводимо детальний аналіз умов конкуренції в галузях за моделлю 5 сил Портера. П’ять сил Портера – це модель, яка ідентифікує та аналізує п’ять конкурентних сил, які формують кожну галузь, і допомагає визначити слабкі та сильні сторони галузі. Аналіз п’яти сил часто використовується для визначення структури галузі та визначення корпоративної стратегії. Наведено в таблиці 6.9.

Таблиця 6.9

Аналіз конкуренції в галузі за Портером

Складові аналізу	Прямі конкуренти в галузі	Потенційні конкуренти в галузі	Постачальники	Клієнти	Товари замінники
	ReconRobotics	Throwbot 2, Cobra MK2	Мінімізація витрат часу для постачальників	Військові та поліцейські	Достатня кількість

Продовження таблиці 6.9

Висновки	Висока конкурентна боротьба	Вийти на ринок можна лише за умови постачання конкурентоздатних умов	Постачальники ставлять велику вартість послуг	Клієнти подають вимоги виробникам	Відсутність локальних товарів для заміни

Обґрунтовуємо фактори конкурентоспроможності в таблиці 6.10.

Таблиця 6.10

Фактори конкурентоспроможності

№	Фактор конкурентоспроможності	Обґрунтування
1.	Універсальність	Можливе використання системи без значних змін
2.	Швидкість системи	Завдяки передавачам та індикаторам розпізнавання шляху відбувається досить швидко
3.	Удосконалення	Система може бути покращена за допомогою оновлень

Тепер перейдемо до аналізу сильних та слабких сторін з точки зору конкурентів та продукту який є дуже важливим у реалізації потенціалу стартап проєкту, його часто використовують для якісного порівняння ситеми. Опис наведений у таблиці 6.11.

Таблиця 6.11

Порівняльний аналіз сильних та слабких сторін

№	Фактор конкурентоспроможності	Бали 1-20	Рейтинг товарів-конкурентів						
			-3	-2	-1	0	1	2	3
1.	Універсальність	15				+			
2.	Швидкість системи	16					+		
3.	Удосконалення	14				+			

Останнім кроком є SWOT-аналіз описаний у таблиці 6.12.

Таблиця 6.12

SWOT-аналіз стартап проєкту

Сильні сторони: швидкість системи, універсальність, можливість покращення	Слабкі сторони: можливі неточності у керуванні
Можливості: великі інвестиції, мала конкуренція	Загрози: невизнання людьми продукту

Після проведеного аналізу розроблено можливості вибору ринкової поведінки та перераховано заходи з їх імплементацією у таблиці 6.13.

Таблиця 6.13

Альтернативи ринкового впровадження стартап проєкту

№	Альтернатива ринку	Ймовірність отримання ресурсів	Строки
1.	Зацікавлення компаній	Продаж проєкту	Кілька років
2.	Пробний період	Клієнт купить про версію	Півтора року
3.	Активна піар кампанія	Популяризація продукту	Рік

6.4 Розроблення ринкової стратегії

Зробимо опис користувачів продукту, що створюється стартап проєктом. Такими групами є військові, поліцейські, звичайні клієнти. Опис наведено в таблиці 6.14.

Таблиця 6.14

Вибір цільових груп потенційних клієнтів

№	Опис профілю цільової групи потенційних клієнтів	Готовність споживачів сприйняти продукт	Орієнтовний попит в межах цільової групи	Інтенсивність конкуренції в сегменті	Простота входу в сегмент
1.	Звичайні клієнти	Мають потребу у використанні таких систем	Попит не є високим	Немає	Вхід простий
2.	Військові	Мають велику потребу у розвідувальних операціях	Попит є високим, через зручність	Є досить висока конкуренція	Вхід складний потрібні сертифікації
3.	Поліція	Мають велику потребу у розвідувальних операціях	Попит є високим, через зручність роботи	Є досить висока конкуренція	Вхід складний так, як потрібні сертифікації
Які цільові групи обрано: військові, поліцейські, звичайні клієнти					

Після результатів аналізу груп користувачів формуємо базову стратегію розвитку. Опис наведений у таблиці 6.15.

Таблиця 6.15

Визначення базової стратегії розвитку

№	Обрана альтернатива розвитку проєкту	Стратегія охоплення ринку	Ключові конкурентоспроможні позиції відповідно до обраної альтернативи	Базова стратегія розвитку
1.	Отримання міжнародних інвестицій	Розробити універсальний продукт	Покращити продукт для певної сфери користувачів	Стратегія диференціації

Після цього наводимо базову стратегію конкурентної поведінки продукту. Опис наведений у таблиці 6.16.

Таблиця 6.16

Визначення базової стратегії конкурентної поведінки

№	Чи є проєкт «Першопрохідцем» на ринку?	Чи буде компанія шукати нових споживачів, або забирати існуючих у конкурентів?	Чи буде компанія копіювати основні характеристики конкурента?	Стратегія конкурентної поведінки
1.	Ні проєкт не є таким	Буде забирати існуючих споживачів у конкурентів	Не буде копіювати	Наступальна

Зробимо опис стратегії позиціонування у результаті опису вимог споживачів та базової стратегії розвитку продукту. Опис наведено у таблиці 6.17.

Таблиця 6.17

Визначення стратегії позиціонування

№	Вимоги до товару цільової аудиторії	Базова стратегія розвитку	Ключові конкурентоспроможні позиції власного стартапу	Вибір асоціацій, які мають сформувати комплексну позицію власного проєкту
1.	Простота роботи, зручне управління,	Стратегія диференціації	Покращити продукт для певної сфери користувачів	Простота, зручність, точність.

6.5 Розроблення маркетингової програми стартап-проєкту

Для початку формуємо маркетингову концепцію товару, який отримає користувач. Для цього треба підвести підсумки висновків зроблених у попередніх аналізах. Дані наведено в таблиці 6.18.

Таблиця 6.18

Визначення ключових переваг концепції потенційного товару

№	Потреба	Вигода, яку пропонує товар	Ключові переваги перед конкурентами
1.	Простота	Має простий інтерфейс	Легкість використання продукту, зі зручним інтерфейсом
2.	Зручність	Забезпечує зручність управління	Зручне використання системи управління продукту
3.	Точність	Забезпечує точність розпізнавання маршруту	Система краще розпізнає маршрут

Розробляємо трирівневу модель товару. Опис наведений у таблиці 6.19.

Таблиця 6.19

Опис трьох рівнів моделі товару

Рівні товару	Сутність та складові
1. Товар за задумом	Швидкий та компактний робот для виконування розвідувальних задач. Зручний та простий інтерфейс управління. Висока точність розпізнавання маршруту, та роботи автономної системи управління дроном для користуача.

Продовження таблиці 6.19

2. Товар у реальному виконанні	Властивості/характеристики	М/Нм	Вр/Тх/Тл/Е/ОР
	1. Точність 2. Компактність 3. Простота 4. Зручність	Нм М М Нм	Тл Ор Вр Тл
	Якість: проведення тестування та перевірки роботи системи		
	Пакування: коробка та диск для інсталяції		
	Марка: назва товару та назва компанії		
3. Товар із підкріпленням	До продажу: консультація для користувача		
	Після продажу: вирішення проблем та помилок		
За рахунок чого потенційний товар буде захищено від копіювання: патенти, авторське право, законодавство.			

Наступним етапом є визначення цінових меж, які треба обрати при встановленні ціни. Опис наведено в таблиці 6.20.

Таблиця 6.20

Визначення меж встановлення цін

№	Рівень цін на товари замінники	Рівень цін на товари аналоги	Рівень доходів цільової групи користувачів	Верхня та нижня межі встановлення ціни на товар/послугу
1.	15000$	20000$	~$50000/міс	$10000-300000

Наступним етапом є обрання системи збуту в межах якої приймається рішення. Обрання системи збуту є дуже важливим аспектом для проєкту. Дані наведено в таблиці 6.21.

Таблиця 6.21

Формування системи збуту

№	Специфіка закупівельної поведінки цільових клієнтів	Функції збуту, які має виконувати постачальник товару	Глибина каналу збуту	Оптимальна система збуту
1.	Разова покупка	Технічна підтримка	Одноріневий	Збут продукту в магазинах

Фінальним кроком є визначення концепції маркетингових комунікацій. Опис наведена в таблиці 6.22.

Таблиця 6.22

Концепція маркетингових комунікацій

№	Специфіка поведінки цільових клієнтів	Канали комунікацій, якими користуютьс я цільові клієнти	Ключові позиції, обрані для позиціонування	Завдання рекламног о повідомле ння	Концепці я рекламно го зверненн я
1.	Покупка товарів через маркетплейс и	Месенджери, соціальні мережі	Простота, зручність, точність	Зацікавити користува ча	Демонстр ація результат ів роботи

Опис концепцій маркетингових комунікацій є дуже важливою частиною опису майбутнього стартап проєкту.

6.6 Висновки до розділу

Створений продукт має певні переваги над конкурентами. Продукт та програма мають шляхи для подальшого розвитку, визначені маркетингові стратегії та шляхи збуту. Основна цільова аудиторія військові.

ВИСНОВКИ

В ході виконання даної магістерської роботи було розроблено адаптивну систему керування автономним роботом.

Проведено дослідження предметної області робототехніки, а саме автономних наземних безпілотників. Основні сценарії розглянуто із можливими альтернативами.

Створено програмний продукт та машину, які дають змогу виконувати певні задачі та проходження заданого маршруту самостійно або за допомогою користувача.

Виконано поставлені задачі, а саме аналіз існуючих систем та методів адаптивного управління роботом, вибрано найкращий метод розробки який підходить для вирішення цієї задачі, розроблено програмне забезпечення для роботи з дроном, та спроєктовано робота.

Машину було запрограмовано на мові програмування C++.

Робота можна використовувати у різних цілях зокрема завданнях розвідки та знаходження певних об’єктів.

В ході роботи були використані такі утиліти для створення програмного забезпечення машини (середовище розробки Arduino IDE, конструктор Arduino, та контролер STM32).

Для роботи з даним дроном не потрібні ніякі програмні забезпечення або комп’ютери, але для роботи з його кодом та зміною конфігурації знадобиться комп’ютер середньої потужності.

Користувачу буде надана можливість керувати роботом для вирішення певних задач та збору інформації, або ж переведення робота в автономний режим для автоматичного створення машиною маршруту.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Малкольм Ф. Що робити, коли машини почнуть робити все? / Ф. Малкольм, Б. Принг. – Київ: Форс, 2014. – 320 с. – (Top Business Awards).
2. Shamau. G. Drone Theory / Gregor Shamau. – London: Penguin, 2015. – 250 с. – (3).
3. Bruk. H. Spying / Henri Bruk. – London: Usborne, 2017. – 80 с. – (Beginners).
4. Army Technology [Електронний ресурс] – Режим доступу до ресурсу: https://www.army-technology.com/.
5. Eca Group [Електронний ресурс] – Режим доступу до ресурсу: https://www.ecagroup.com/.
6. Encarnação P. Robotic Assistive Technologies / P. Encarnação, A. Cook. – London: CRC Press, 2017. – 382 с. – (2).
7. Глазунов А. Нові механізми в новій робототехніці / Артем Глазунов. – Київ: Техносфера, 2021. – 316 с.
8. Білов А. Мікроконтроллери AVR / Анатолій Білов. – Київ: Наука і техніка, 2020. – 544 с.
9. Gurstelle W. Building Bots. Designing and Building Warrior Robots / Wictor Gurstelle. – London: Usborne, 2002. – 443 с. – (2).
10. Cook D. Intermediate Robot Building / David Cook. – New York: Usborne, 2010. – 320 с. – (3).
11. Margolis M. Make an Arduino-Controlled Robot / Michael Margolis. – York: Penguin, 2012. – 304 с. – (1).
12. Cook D. Robot Building for Beginners / David Cook. – London: Usborne, 2009. – 602 с. – (2).
13. Воротніков О. Інформаційні пристрої робототехнічних систем / Олександр Воротніков. – Київ: Вільямс, 2005. – 320 с. – (2). – (Top Business Awards).

14. Смірнов А. Мехатроніка та робототехніка. Системи мікропереміщень із п'єзоелектричними приводами. Навчальний посібник / Аркадій Смірнов. – Київ: Вільямс, 2003. – 352 с. – (2).
15. Юрічев А. Основи робототехніки / Артем Юрічев. – Київ: Вільямс, 2012. – 250 с. – (3).
16. Потапов В. Пилотажні радіокеровані моделі літаків / Володимир Потапов. – Київ: ДМК-Пресс, 2014. – 352 с. – (3).
17. Предко М. 123 експерименти з робототехніки / Максим Предко. – Київ: Вільямс, 2015. – 304 с. – (2).
18. Вільямс Д. Програмовані роботи. Створюємо робота для своєї домашньої майстерні / Джордж Вільямс. – Київ: Форс, 2014. – 602 с. – (3).
19. Nikel A. Robots mobiles programmables Techniques avancees / Alex Nikel. – London: Usborne, 2017. – 250 с. – (2).
20. Брага С. Створення роботів у домашніх умовах / Скотт Брага. – Київ: Вільямс, 2012. – 320 с. – (3).
21. Предко М. Пристрої керування роботами. Схемотехніка та програмування / Максим Предко. – Київ: ДМК-Пресс, 2013. – 304 с. – (2).
22. Мейєрс С. Ефективний та сучасний C++ / Скотт Мейєрс. – Київ: Вільямс, 2015. – 304 с.
23. Шилдт Г. C++. Базовий курс / Герберт Шилдт. – Київ: Вільямс, 2015. – 602 с. – (3).
24. Гантерот К. Оптимізація програм на C++ / Курт Гантерот. – Київ: Вільямс, 2013. – 400 с.
25. О'Двайр А. Опановуємо C++ / Артур О'Двайр. – Київ: ДМК-Пресс, 2012. – 352 с.
26. Петін В. 77 пректів для Arduino / Віктор Петін. – Київ: ДМК-Пресс, 2012. – 352 с.

27. Ranberg D. Arduino for inventors / Derek Ranberg. – London: Usborne, 2014. – 288 с.
28. Norris D. Programming with STM32: Getting Started with the Nucleo Board and C/C++ / Donald Norris. – Washington: McGraw Hill TAB, 2018. – 304 с.
29. Arduino Education [Електронний ресурс] – Режим доступу до ресурсу: https://www.arduino.cc/.
30. Knight S. Arduino for Beginners: Step-by-Step Guide to Arduino / Simon Knight. – Washington: Independently published, 2012. – 139 с.

ДОДАТОК А

Адаптивне управління автономним роботом на базі контролера STM32

Апробація

MODERN RESEARCH IN WORLD SCIENCE

Аркушів 7

Київ — 2022

SCI-CONF.COM.UA

MODERN RESEARCH IN WORLD SCIENCE



PROCEEDINGS OF IX INTERNATIONAL SCIENTIFIC AND PRACTICAL CONFERENCE NOVEMBER 28-30, 2022

LVIV
2022

117. *Лесик М. О., Крячок О. С.* 525
АДАПТИВНЕ УПРАВЛІННЯ АВТОНОМНИМ РОБОТОМ НА БАЗІ КОНТРОЛЕРА STM32
118. *Летута Т. М., Карбівнича Т. В., Пенкіна Н. М.* 529
АНАЛІЗ МІЖНАРОДНИХ ВИМОГ ДО ЯКОСТІ ТА БЕЗПЕЧНОСТІ ХАРЧОВИХ ПРОДУКТІВ
119. *Лук'янчиков Г. О.* 536
ВИКОРИСТАННЯ ОДНОЕТАПНИХ ДЕТЕКТОРІВ ДЛЯ ОЦІНКИ ПОЗИ ГОЛОВИ ЛЮДИНИ
120. *Нельга А. Т., Самохвалов С. П., Тітов І. В., Щербина В. В.* 541
МОНІТОРИНГ ФУНКЦІОНАЛЬНОГО СТАНУ СПОРТСМЕНІВ З ВИКОРИСТАННЯМ БЕЗДРОТОВОЇ МЕРЕЖІ
121. *Обіход Т. В., Біленчук П. Д.* 545
КІБЕРБЕЗПЕКА УКРАЇНИ І ЗАСОБИ ЇЇ РЕАЛІЗАЦІЇ
122. *Олійник Д. О., Кузьменко І. М.* 552
ІНТЕЛЕКТУАЛЬНИЙ АНАЛІЗ ДАНИХ ЗІ СТАНУ ДОВКІЛЛЯ НАВКОЛО ЧАЕС З ВИКОРИСТАННЯМ МОВИ PYTHON
123. *Ольшевська О. В., Дерменжи Т. С.* 555
ДОСЛІДЖЕННЯ ПРОЦЕСУ АВТОМАТИЗАЦІЇ ДІЯЛЬНОСТІ ВІДДІЛУ КАДРІВ ОНТУ
124. *Петров В. М., Кирієнко М. А.* 558
ПРОЕКТУВАННЯ ТА ДОСЛІДЖЕННЯ СТЕНДУ ДЛЯ КЕРМОВИХ ПІДСИЛЮВАЧІВ
125. *Пінчук Є. Р., Нудченко Л. А., Черняк Л. П.* 562
ПИТАННЯ ВИГОТОВЛЕННЯ ФОРМУВАЛЬНОГО ГІПСУ
126. *Проказа У. С.* 565
ПРОБЛЕМИ БЕЗПЕКИ ХМАРНИХ ДАНИХ
127. *Рибалко Р. О.* 569
АНАЛІЗ ТЕХНОЛОГІЙ СКЛАДАННЯ ТА ПАКУВАННЯ BGA-КОМПОНЕНТІВ
128. *Рибчак З. Л., Павленко К. К.* 576
ПРОЄКТ СИСТЕМИ ВИГОТОВЛЕННЯ ТА ЗАМОВЛЕННЯ ЮВЕЛІРНИХ ПРИКРАС
129. *Соколенко В. В., Кобзар О. С.* 579
РОЗРОБКА РЕЦЕПТУРИ ПЕЛЬМЕНІВ З ЛЮПИНОВИМ БОРОШНОМ
130. *Степанов Д. М., Горобець І. А.* 583
ПОРІВНЯЛЬНА ОЦІНКА МЕХАНІЧНОЇ МІЦНОСТІ СКЛОПЛАСТИКОВИХ СТЕРЖНІВ СИЛОВОГО ЕЛЕМЕНТУ ОПТИЧНОГО КАБЕЛЮ
131. *Струтинський В. Б., Нога І. С., Гольцов Б. О.* 588
ОКРЕМІ ПРОБЛЕМИ ПРОЕКТУВАННЯ МАНІПУЛЯТОРІВ НАЗЕМНИХ РОБОТИЗОВАНИХ КОМПЛЕКСІВ

АДАПТИВНЕ УПРАВЛІННЯ АВТОНОМНИМ РОБОТОМ НА БАЗІ КОНТРОЛЕРА STM32

Лесик Максим Олександрович
Студент 6 курсу
Науковий керівник:
Крячок Олександр Степанович
к.т.н., доцент
Національний технічний університет України
Київський політехнічний інститут ім. Ігоря Сікорського
м. Київ, Україна
mlesyk42@gmail.com

Вступ. З розвитком технологій роботи стали життєво важливим інструментом у повсякденному житті людей. Важливим аспектом створення роботів є те, що вони корисні та ефективні. Вони допомагають зробити життя людей легшим, простішим і комфортнішим. Вони також замінюють людську робочу силу для виконання нудної, монотонної, брудної чи небезпечної роботи. Вони привели людство вперед, будучи частиною відкриття нової інформації. Допомога у військових завданнях, надання допомоги учням і вчителям у навчанні та догляд за літніми людьми є одними з ролей, які роботи виконують у повсякденному житті.

Мета роботи. Метою розробки роботизованої системи є створення програмного продукту та машини, що дасть змогу, виконувати певні розвідувальні задачі та проходження певного маршруту самостійно або за допомогою користувача.

Матеріали та методи. Безпілотний наземний транспортний засіб — це комплексна інтелектуальна система, яка об'єднує сприйняття навколишнього середовища, місцезнаходження, навігацію, планування шляху, прийняття рішень і керування рухом. Він поєднує високі технології, включаючи інформатику, злиття даних, машинне бачення, [1] глибоке навчання тощо, щоб задовольнити реальні потреби для досягнення заздалегідь визначених цілей.

У сфері цивільного застосування безпілотні наземні транспортні засоби в основному втілені в автономному водінні. Високоінтелектуальні моделі водіїв можуть повністю або частково замінити активний контроль водія. Крім того, безпілотні

наземні транспортні засоби з датчиками можуть легко діяти як «транспортні засоби-зонди» та здійснювати зондування дорожнього руху для кращого обміну інформацією з іншими агентами в інтелектуальних транспортних системах. Таким чином, він має великий потенціал у зменшенні дорожньо-транспортних пригод і зменшенні заторів. У сфері військового застосування він компетентний у таких завданнях, як отримання розвідданих, моніторинг і розвідка, транспортування та матеріально-технічне забезпечення, розмінування та розміщення саморобних вибухових пристроїв, забезпечення вогневої підтримки, передача зв'язку та медична передача на полі бою, що може ефективно допомагати військам у бойових діях.

Технологія сприйняття середовища включає одночасну локалізацію та відображення, семантичну сегментацію, виявлення транспортних засобів, виявлення пішоходів, виявлення дороги та багато інших аспектів. Серед різноманітних технологій, оскільки транспортні засоби є найчисленнішими та найрізноманітнішими цілями в середовищі водіння, як правильно ідентифікувати транспортні засоби, стало гарячою точкою дослідження для безпілотних наземних транспортних засобів. У цивільній сфері правильне виявлення дорожніх транспортних засобів може зменшити кількість дорожньо-транспортних пригод, створити більш повну картину безпеки і досягти кращої інтеграції з моделлю водія,[2] тоді як у військовій сфері правильне виявлення цілей військових транспортних засобів має велике значення для поля бою. розвідка, оцінка загроз і точна атака в сучасній війні.

Результати та обговорення. Для розробки системи управління роботом була використана мова програмування C++ та Arduino Ino. Для проектування робота був використаний контролер STM32 та конструктор Arduino(рисунок 1).

Завдання цього проекту полягає в тому, щоб дати змогу звичайному користувачу ознайомитись з поняттям роботизована автономна система управління та її роботою в різних галузях та задачах.

В результаті розробки продукту, користувачу буде надана можливість керувати роботом для вирішення певних задач та збору інформації, або ж переведення робота в автономний режим для автоматичного створення машиною маршруту.

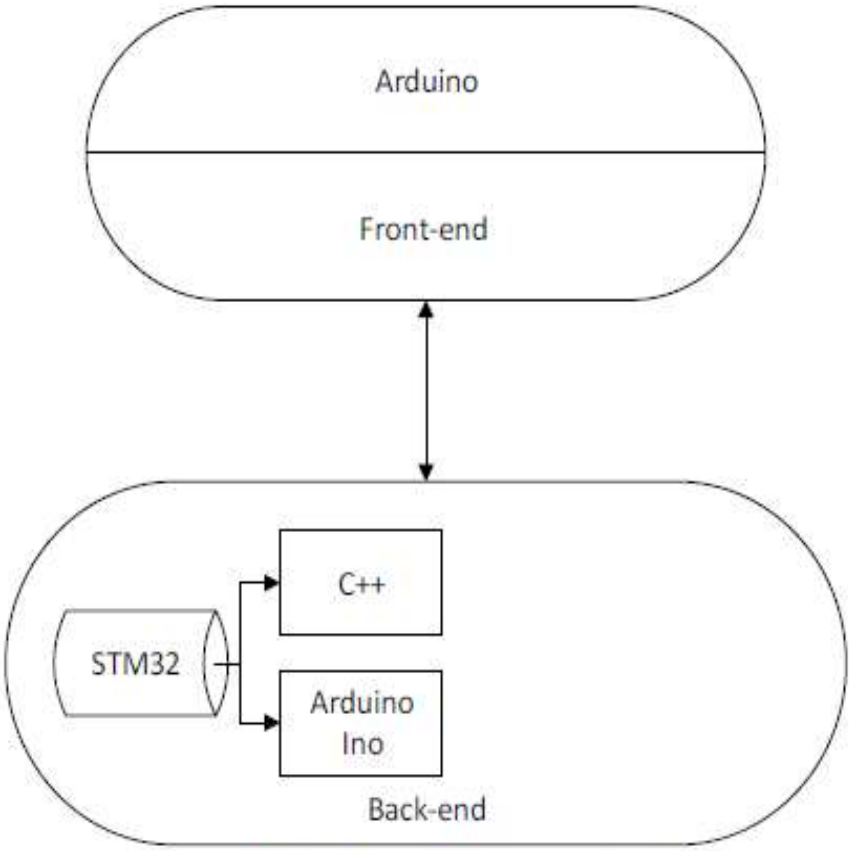


Рисунок 1– Структура системи

Програма надає користувачу різні функціональні можливості:

- керування роботом;
- перевід робота в автоматичний режим;
- перегляд даних по маршруту.

Вхідними даними для системи є показання з датчиків робота, які він збирає в процесі своєї роботи та прокладанню маршруту.

Вихідними даними є оброблена інформація про маршрут та самостійне або кероване переміщення робота у просторі.

Висновки. В ході виконання даної роботи було розроблено адаптивну систему керування автономним роботом.

Робота можна використовувати у різних цілях зокрема завданнях розвідки та знаходження певних об'єктів.

Користувачу буде надана можливість керувати роботом для вирішення певних задач та збору інформації, або ж переведення робота в автономний режим для автоматичного створення машиною маршруту.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Shamau. G. Drone Theory / Gregor Shamau. – London: Penguin, 2015. – 250 c. – (3).
2. Bruk. H. Spying / Henri Bruk. – London: Usborne, 2017. – 80 c. – (Beginners).